# LAPORAN MILESTONE 1 TUGAS BESAR IF3070

# DASAR INTELIGENSI ARTIFISIAL

## Penerapan Algoritma Local Search pada Penataan Kendaraan di Geladak Kapal Kargo


Disusun oleh Kelompok 18 - Triplets:

| | |
|---|---|
| Indah Ramadhani | 18224037 |
| Tiara Clianta Andiwi | 18224078 |
| Salsabila Shofiyah | 18224088 |

**SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA**

**INSTITUT TEKNOLOGI BANDUNG**


**2026**

# DAFTAR ISI

**DAFTAR ISI** ..... **2**
**DAFTAR TABEL** ..... **4**
**DAFTAR GAMBAR** ..... **5**
**BAB I** ..... **6**
**DESKRIPSI PERSOALAN** ..... **6**
1.1 Latar Belakang ..... 6
1.2 Deskripsi Persoalan ..... 6
1.3 Tujuan Penyelesaian ..... 7
**BAB II** ..... **8**
**PEMBAHASAN** ..... **8**
2.1 Pemodelan Persoalan ..... 8
2.1.1 Dataset dan Karakteristik Kendaraan ..... 8
2.1.2 Representasi State ..... 9
2.1.3 Constraint Penempatan ..... 9
2.1.4 Pembentukan Initial State dan Neighbor ..... 11
2.2 Pemilihan Objective Function ..... 12
2.3 Skenario dan Parameter Pengujian ..... 13
2.4 Steepest Ascent Hill-Climbing ..... 14
2.5 Simulated Annealing ..... 18
2.6 Genetic Algorithm ..... 25
2.7 Perbandingan Antaralgoritma ..... 32
2.8 Visualisasi dan Program Demo ..... 33
**BAB III** ..... **37**
**IMPLEMENTASI BONUS** ..... **37**
3.1 Variasi Hill-Climbing ..... 37
3.1.1 Stochastic Hill-Climbing ..... 37
3.1.2 Hill-Climbing with Sideways Move ..... 41
3.1.3 Random Restart Hill-Climbing ..... 45
3.1.4 Analisis Perbandingan Variasi Hill-Climbing ..... 49
3.2 Objective Function ETA ..... 50
3.2.1 Rancangan Objective Function ETA ..... 50
3.2.2 Hasil dan Analisis ..... 51
3.3 Multi-Kapal ..... 53
3.3.1 Implementasi Multi-Kapal ..... 53
3.3.2 Hasil dan Analisis ..... 53
**BAB IV** ..... **58**
**KESIMPULAN DAN SARAN** ..... **58**
4.1 Kesimpulan ..... 58
4.2 Saran ..... 58
**BAB V** ..... **60**
**PEMBAGIAN TUGAS ANGGOTA KELOMPOK** ..... **60**

**REFERENSI** ..... **61**
**LAMPIRAN** ..... **62**
Lampiran A Form Penggunaan Generative AI ..... 62
Lampiran B Tautan Repository GitHub ..... 63
Lampiran C Bukti Pengerjaan ..... 63

# DAFTAR TABEL

Tabel 2.1 Karakteristik jenis kendaraan..... 8
Tabel 2.2 Kasus uji..... 8
Tabel 2.3 Parameter pengujian.....13
Tabel 2.4 Hasil Steepest Ascent Hill-Climbing..... 15
Tabel 2.5 Hasil Simulated Annealing.....22
Tabel 2.6 Hasil Genetic Algorithm percobaan A dengan populasi 50.....29
Tabel 2.7 Hasil Genetic Algorithm percobaan B dengan iterasi 200.....29
Tabel 2.8 Perbandingan hasil ketiga algoritma.....32
Tabel 2.9 Fungsi visualisasi..... 33
Tabel 3.1 Hasil Stochastic Hill-Climbing..... 38
Tabel 3.2 Hasil Hill-Climbing with Sideways Move..... 43
Tabel 3.3 Hasil Random Restart Hill-Climbing.....46
Tabel 3.4 Ringkasan keempat varian Hill-Climbing..... 49
Tabel 3.5 Hasil objective function ETA..... 51
Tabel 3.6 Hasil kasus multi-kapal..... 53
Tabel 3.7 Hasil Hill-Climbing pada kasus multi-kapal.....54
Tabel 5.1 Pembagian Tugas.....60

# DAFTAR GAMBAR

Gambar 2.1 Visualisasi awal Steepest Ascent Hill-Climbing..... 16
Gambar 2.2 Visualisasi akhir Steepest Ascent Hill-Climbing.....17
Gambar 2.3 Grafik Steepest Ascent Hill-Climbing..... 17
Gambar 2.4 State awal dan state akhir percobaan 1 yang mencapai nilai 296..... 23
Gambar 2.5 Grafik nilai Objective Function ketiga percobaan.....24
Gambar 2.6 Grafik peluang terhadap iterasi percobaan 1..... 24
Gambar 2.7 State awal dan state akhir percobaan yang mencapai nilai 308,.....30
Gambar 2.8 Grafik nilai maksimum dan rata-rata populasi,..... 31
Gambar 2.9 Tampilan menu awal..... 34
Gambar 2.10 Tampilan hasil di terminal dan pop up visualisasi.....34
Gambar 2.11 Contoh visualisasi dua kapal.....36
Gambar 3.1 Visualisasi awal Stochastic Hill-Climbing..... 39
Gambar 3.2 Visualisasi akhir Stochastic Hill-Climbing..... 40
Gambar 3.3 Grafik Stochastic Hill-Climbing..... 40
Gambar 3.4 Visualisasi awal Sideways Hill-Climbing.....43
Gambar 3.5 Visualisasi akhir Sideways Hill-Climbing..... 44
Gambar 3.6 Grafik Sideways Hill-Climbing.....44
Gambar 3.7 Visualisasi awal Random Restart Hill-Climbing.....47
Gambar 3.8 Visualisasi akhir Random Restart Hill-Climbing..... 48
Gambar 3.9 Grafik Random Restart Hill-Climbing.....48
Gambar 3.10 Grafik perbandingan keempat varian Hill-Climbing..... 49
Gambar 3.11 State akhir Genetic Algorithm dengan objective function ETA..... 52
Gambar 3.12 Grafik perbandingan varian Hill-Climbing dengan objective function ETA..... 53
Gambar 3.13 State akhir Hill-Climbing pada kasus multi-kapal.....55
Gambar 3.14 Grafik nilai objective function HC pada multi-kapal.....55
Gambar 3.15 State akhir Simulated Annealing pada kasus multi-kapal..... 56
Gambar 3.16 Grafik nilai objective function SA ketiga percobaan.....56
Gambar 3.17 State akhir Genetic Algorithm pada kasus multi-kapal..... 57
Gambar 3.18 Grafik nilai objective function GA pada multi-kapal.....57

# BAB I
# DESKRIPSI PERSOALAN

## 1.1 Latar Belakang

Penataan kendaraan pada geladak kapal kargo harus memperhatikan beberapa hal, terutama luas ruang yang tersedia dan kapasitas berat kapal. Setiap kendaraan memiliki ukuran, berat, biaya pengiriman (Shipping Fee), serta estimasi waktu kedatangan (Estimated Time of Arrival) yang berbeda. Di sisi lain, kapal memiliki batas untuk ukuran geladak dan kapasitas berat maksimum jumlah kendaraan yang dapat diangkut. Kondisi ini membuat penatan kendaraan tidak dapat dilakukan hanya dengan mempertimbangkan jumlah kendaraan, tetapi juga memperhatikan kombinasi dan posisi penempatannya.

Dalam proses penataan, setiap kendaraan harus ditempatkan pada posisi dan orientasi yang sesuai agar tidak saling tumpang tindih (overlap) maupun melewati batas geladak. Total berat kendaraan yang diangkut juga tidak boleh melebihi kapasitas kapal. Dengan berbagai batasan tersebut, terdapat banyak kemungkinan susunan kendaraan yang dapat dibentuk, tetapi tidak semuanya menghasilkan biaya pengiriman yang sama. Oleh karena itu, diperlukan cara untuk mencari susunan kendaraan yang dapat memberikan total Shipping Fee setinggi mungkin tanpa melanggar batasan yang berlaku.

Salah satu pendekatan yang dapat digunakan untuk menyelesaikan permasalahan ini adalah local search. Pendekatan ini berkerja dengan mencari solusi yang lebih baik melalui perubahan terhadap suatu keadaan awal (initial state). Pada tugas besar ini, algoritma yang digunakan meliputi Hill-Climbing, Simulated Annealing, dan Genetic Algorithm. Ketiganya memiliki cara pencarian yang berbeda sehingga menarik untuk dibandingkan, baik dari nilai solusi yang dihasilkan maupun proses pencariannya.

## 1.2 Deskripsi Persoalan

Pada tugas besar ini, permasalahan yang diselesaikan berkaitan dengan penataan kendaraan pada geladak kapal kargo. Pengembang berperan sebagai Forward Deployed Engineer (FDE) yang bertugas membuat sistem untuk menentukan kendaraan yang akan diangkut beserta posisi dan orientasi penempatannya. Setiap kendaraan memiliki atribut berupa lebar (width), panjang (length), berat (weight), ShippingFee, dan ETA. Sementara itu, setiap kapal memiliki geladak berbentuk grid dua dimensi dengan ukuran dan kapasitas berat tertentu.

Kendaraan dapat ditempatkan secara horizontal maupun vertikal, bergantung pada ruang yang tersedia. Kendaraan juga dapat tidak diangkut apabila tidak memungkinkan untuk dimasukkan ke dalam susunan. Dalam menentukan penempatan, terdapat tiga batasan utama yang harus dipenuhi. Kendaraan tidak boleh saling tumpang tindih, seluruh bagian kendaraan harus berada di dalam batas geladak, dan total berat kendaraan yang diangkut tidak boleh melebihi kapasitas

maksimum kapal. Suatu susunan dianggap valid apabila ketiga batasan tersebut terpenuhi.

Tujuan utama dari permasalahan ini adalah memperoleh susunan kendaraan dengan total ShippingFee tertinggi. Selain itu, digunakan fungsi objektif tambahan berbasis ETA untuk mempertimbangkan posisi kendaraan yang perlu diprioritaskan saat pembongkaran. Kendaraan dengan ETA yang lebih mendesak diharapkan berada di dekat pintu bongkar (ramp) agar lebih mudah diakses. Dengan demikian, susunan kendaraan dapat dievaluasi berdasarkan nilai ShippingFee maupun kesesuaiannya dengan prioritas ETA.

Pencarian solusi dilakukan menggunakan algoritma Hill-Climbing, Simulated Annealing, dan Genetic Algorithm. Prosesnya dimulai dari sebuah initial state yang dibangkitkan secara acak. Untuk menghasilkan susunan lain, digunakan tiga jenis perubahan, yaitu menukar dua kendaraan, memindahkan kendaraan ke posisi lain, dan mengubah orientasi kendaraan. Setiap perubahan tetap harus memenuhi batasan penempatan yang telah ditentukan. Melalui proses tersebut, masing-masing algoritma mencari susunan dengan nilai fungsi objektif yang lebih baik.

Selain pengujian pada satu kapal, tugas besar ini juga mencakup pengembangan untuk permasalahan multi-kapal dan penerapan beberapa variasi Hill-Climbing. Pengembangan ini dilakukan untuk melihat bagaimana metode pencarian dapat diterapkan pada kondisi yang berbeda, baik dari jumlah kapal yang tersedia maupun strategi pencarian solusi.

## 1.3 Tujuan Penyelesaian

Tugas besar ini bertujuan untuk menerapkan dan membandingkan algoritma *local search* dalam menyelesaikan permasalahan penataan kendaraan pada geladak kapal kargo. Adapun tujuan yang ingin dicapai adalah sebagai berikut:

1. Merepresentasikan penataan kendaraan sebagai suatu *state* yang memuat informsi kendaraan yang daingkut, posisi, dan orientasinya.
2. Mengimplementasikan fungsi objektif dan pemeriksaan batasan untuk memastikan setiap susunan kendaraan memenuhi ketentuan penempatan serta kapasitas kapal.
3. Menimplementasikan algoritma Hill-Climbing, Simulated Annealing, dan Genetic Algorithm untuk mencari susunan kendaraan dengan total ShippingFee yang tinggi.
4. Membandingkan hasil ketiga algoritma berdasarkan nilai fungsi objektif, jumlah iterasi, dan waktu komputasi.
5. Menganalisis penggunaan fungsi objektif tambahan berbasis ETA dalam menentukan penempatan kendaraan yang perlu diprioritaskan.
6. Mengembangkan variasi Hill-Climbing dan menetapkan optimasi pada skenario mulit-kapal.

# BAB II
# PEMBAHASAN

## 2.1 Pemodelan Persoalan

### 2.1.1 Dataset dan Karakteristik Kendaraan

Dataset yang digunakan dibuat sendiri oleh kelompok dan disimpan dalam bentuk file JSON. Setiap file berisi daftar kapal dan daftar kendaraan. Data kapal berisi lebar, panjang, dan kapasitas berat, sedangkan data kendaraan berisi lebar, panjang, berat, ShippingFee, dan ETA. Kendaraan dibagi menjadi empat jenis seperti pada tabel berikut.

**Tabel 2.1 Karakteristik jenis kendaraan**

| Jenis | Ukuran | Berat | ShippingFee |
|---|---|---|---|
| Motor | 1 × 2 | 1 | 10 |
| Sedan | 2 × 3 | 2 | 30 |
| SUV | 2 × 4 | 3 | 38 |
| Truk | 3 × 4 | 5 | 70 |

Nilai ShippingFee sengaja tidak dibuat sebanding dengan ukuran dan berat kendaraan. Sedan paling menguntungkan jika dilihat dari beratnya, sedangkan truk paling menguntungkan jika dilihat dari luasnya. Dengan begitu, tidak ada satu jenis kendaraan yang selalu paling baik untuk dipilih, sehingga algoritma harus mencari kombinasi kendaraan yang paling tepat. Nilai ETA setiap kendaraan berada di antara 1 sampai 10 hari, dan kendaraan dengan ETA paling lama 3 hari dianggap mendesak.

Terdapat dua kasus uji yang digunakan, yaitu kasus satu kapal dan kasus multi-kapal.

**Tabel 2.2 Kasus uji**

| Kasus | Kapal | Jumlah kendaraan | Nilai optimum |
|---|---|---|---|
| kasus_satu_kapal | 1 kapal 8 × 8, kapasitas 22 | 13 | 308 |
| kasus_multi_kapal | 1 kapal 8 × 8 kapasitas 22 dan<br>1 kapal 6 × 8 kapasitas 16 | 20 | 524 |

Pada kedua kasus, total berat dan luas seluruh kendaraan lebih besar dari kapasitas kapal, sehingga pasti ada kendaraan yang tidak terangkut. Nilai optimum pada tabel dihitung dari kombinasi kendaraan terbaik yang masih muat secara berat dan luas. Nilai ini digunakan untuk melihat seberapa dekat hasil setiap algoritma dengan solusi terbaik. Kasus multi-kapal hanya digunakan pada bagian bonus.

### 2.1.2 Representasi State

State adalah susunan seluruh kendaraan pada suatu waktu. State disimpan dalam bentuk tuple yang panjangnya sama dengan jumlah kendaraan. Setiap isi tuple akan menyimpan posisi satu kendaraan, yang berisi nomor kapal, koordinat x, koordinat y, dan orientasinya (horizontal atau vertikal). Koordinat x mengikuti lebar kapal, koordinat y mengikuti panjang kapal dan pintu masuk kapal berada di sisi y = 0. Orientasi H berarti kendaraan diletakkan sesuai ukuran aslinya, sedangkan V berarti kendaraan diputar. Kendaraan yang tidak diangkut (berada di luar kapal) ditandai dengan nomor kapal -1.

```
((0, 0, 0, "H"),
(-1, 0, 0, "H"),
(0, 2, 4, "V"),
...)
```

Pada contoh di atas, kendaraan pertama berada di kapal pertama pada koordinat 0,0 dengan orientasi horizontal, kendaraan kedua tidak diangkut, dan kendaraan ketiga berada di koordinat 2,4 dengan orientasi vertikal. Bentuk state ini juga bisa digunakan untuk kasus multi-kapal karena nomor kapalnya ikut disimpan di state.

### 2.1.3 Constraint Penempatan

Semua *constraint* diperiksa oleh fungsi cek_placement di src/core.py. Fungsi ini yang mengubah posisi setiap kendaraan menjadi daftar *cells* yang ditempatinya, lalu memeriksa tiga hal, yaitu:

1. Semua sel kendaraan harus berada di dalam kapal
2. Sel yang ditempati suatu kendaraan tidak boleh *overlap* dengan kendaraan lain.
3. Total berat kendaraan di setiap kapal tidak boleh melebihi kapasitasnya.

Apabila salah satu syarat tidak terpenuhi, *state* dianggap tidak valid. Setiap *state* baru yang dibuat oleh ketiga algoritma elalu diperiksa terlebih dahulu dengan fungsi ini, sehingga semua state yang digunakan sudah pasti valid.

```python
def cek_placement(problem, state):
    ships = problem["ships"]
    vehicles = problem["vehicles"]

    if not isinstance(state, (tuple, list)):
        return False
    if len(state) != len(vehicles):
        return False

    used_cells = []
    ship_weights = []
    for ship in ships:
        used_cells.append(set())
        ship_weights.append(0)

    for i in range(len(vehicles)):
        vehicle = vehicles[i]
        placement = state[i]

        if not isinstance(placement, (tuple, list)):
            return False
        if len(placement) != 4:
            return False

        ship_idx, x, y, orientation = placement

        if type(ship_idx) != int:
            return False
        if type(x) != int or type(y) != int:
            return False
        if orientation not in ("H", "V"):
            return False

        if ship_idx == -1:
            if tuple(placement) != OUT:
                return False
            continue

        if ship_idx < 0 or ship_idx >= len(ships):
            return False
        if x < 0 or y < 0:
            return False

        ship = ships[ship_idx]
        cells = posisi_vehicle(vehicle, placement)
        if cells is None:
            return False

        for cx, cy in cells:
```

```
            if cx >= ship["w"] or cy >= ship["l"]:
                return False

        if len(cells.intersection(used_cells[ship_idx])) > 0:
            return False
        used_cells[ship_idx].update(cells)

        ship_weights[ship_idx] += vehicle["weight"]
        if ship_weights[ship_idx] > ship["capacity"]:
            return False

    return True
```

### 2.1.4 Pembentukan Initial State dan Neighbor

Initial state dibuat oleh fungsi random_state. Urutan kendaraan akan diacak terlebih dahulu, kemudian setiap kendaraan dicoba diletakkan di posisi dan orientasi acak. Posisi pertama yang valid akan langsung digunakan. Jika tidak ada posisi yang valid, kendaraan tersebut tetap berada diluar kapal, sehingga hasil state acak sudah pasti valid.

Neighbor adalah state baru yang dibentuk dari state sekarang dengan satu langkah perubahan (move). Terdapat tiga langkah yang digunakan.

1. Swap, yaitu menukar posisi dua kendaraan.
2. Move, yaitu memindahkan satu kendaraan ke posisi lain, termasuk memasukkan kendaraan ke kapal atau mengeluarkannya dari kapal.
3. Rotate, yaitu mengubah orientasi kendaraan dari horizontal ke vertikal dan sebaliknya.

Neighbor ini dapat diambil dengan dua cara. Fungsi all_neighbors digunakan untuk membuat seluruh neighbor yang valid dan dipakai oleh Hill-Climbing. Fungsi random_neighbor digunakan untuk mengambil neighbor valid secara acak, dipakai oleh Simulated Annealing dan proses mutasi pada Genetic Algorithm.

all_neighbor dari file src/neighbor.py

```
def all_neighbors(problem, state):
    vehicles = problem["vehicles"]
    ships = problem["ships"]
    n = len(state)
    kandidat = []

    # swap
    for i in range(n):
        for j in range(i + 1, n):
            if state[i][0] == -1 and state[j][0] == -1:
```

```
                continue
            if cek_samevehicle(vehicles[i], vehicles[j]):
                continue
            kandidat.append(action_move(state, ("swap", i, j)))

    for i in range(n):
        v = vehicles[i]
        di_dalam = state[i][0] != -1

        # rotate
        if di_dalam:
            kandidat.append(action_move(state, ("rotate", i)))
        # move keluar kapal
        if di_dalam:
            kandidat.append(action_move(state, ("move", i, OUT)))
        # move ke semua posisi
        daftar_orient = [state[i][3]] if di_dalam else ["H", "V"]
        for o in daftar_orient:
            h, wd = ukuran_vehicle(v, o)
            for s, ship in enumerate(ships):
                for r in range(ship["w"] - h + 1):
                    for c in range(ship["l"] - wd + 1):
                        p = (s, r, c, o)
                        if p != state[i]:
                            kandidat.append(action_move(state, ("move", i,
p)))

    return [k for k in kandidat if cek_placement(problem, k)]
```

random_neighbor dari file src/neighbor.py

```
def random_neighbor(problem, state, max_coba=100):
    for _ in range(max_coba):
        move = random_move(problem, state)
        if move is None:
            continue
        nxt = action_move(state, move)
        if cek_placement(problem, nxt):
            return nxt
    return None
```

## 2.2 Pemilihan Objective Function

Objective function yang digunakan adalah total ShippingFee dari seluruh kendaraan yang diangkut, sesuai dengan spesifikasi. Fungsi value_fee

menjumlahkan ShippingFee setiap kendaraan yang berada dalam kapal, sedangkan kendaraan yang tidak diangkut tidak ikut dihitung. Semakin besar nilainya, maka semakin baik susunan kendaraan tersebut.

Batasan penempatan tidak dimasukkan ke dalam objective function dalam bentuk penalti. Hal ini karena setiap state sudah diperiksa oleh fungsi cek_placement sebelum dinilai, sehingga state yang tidak valid tidak pernah digunakan.

```
def value_fee(problem, state):
    vehicles = problem["vehicles"]
    total_fee = 0
    for i in range(len(vehicles)):
        placement = state[i]
        if placement[0] != -1:
            total_fee += vehicles[i]["fee"]
    return total_fee
```

## 2.3 Skenario dan Parameter Pengujian

Eksperimen wajib dijalankan pada kasus satu kapal dengan objective function ShippingFee. Setiap algoritma dijalankan sebanyak tiga kali dengan seed yang tetap, sehingga hasilnya dapat diulang dan selalu memberikan angka yang sama. Hal yang dicatat pada setiap percobaan adalah state awal dan akhir, nilai objective function, jumlah iterasi, durasi, dan grafik nilai objective function terhadap iterasi.

**Tabel 2.3 Parameter pengujian**

| Algoritma | Parameter | Seed |
|---|---|---|
| Steepest Ascent Hill-Climbing | Iterasi maksimal 500 | 42, 43, 44 |
| Simulated Annealing | Suhu awal 100, alpha 0,9998, suhu minimal 0,1 | 11, 22, 33 |
| Genetic Algorithm percobaan A | Populasi 50, iterasi 100, 200, dan 400 | 1, 2, 3 |
| Genetic Algorithm percobaan B | Iterasi 200, populasi 20, 50, dan 100 | 4, 5, 6 |

Pada Genetic Algorithm, percobaan A menggunakan jumlah populasi sebagai kontrol, sedangkan percobaan B menggunakan jumlah iterasi sebagai kontrol, sesuai dengan spesifikasi. Peluang crossover sebesar 0,9 dan peluang mutasi sebesar 0,3 digunakan pada semua percobaan.

Seed setiap algoritma berbeda karena masing-masing dijalankan oleh anggota yang berbeda. Akibatnya, state awal ketiga algoritma tidak sama, sehingga perbandingan pada subbab 2.7 lebih tepat dibaca sebagai gambaran umum. Parameter untuk eksperimen bonus dijelaskan pada BAB III.

Eksperimen Genetic Algorithm dijalankan menggunakan Python 3.13.0 pada laptop MacBook Pro dengan chip Apple M3, RAM 8 GB, dan sistem operasi macOS 26.2. Eksperimen Hill-Climbing dan Simulated Annealing dijalankan pada laptop anggota lainnya, sehingga durasi antaralgoritma dapat sedikit dipengaruhi oleh perbedaan perangkat.

## 2.4 Steepest Ascent Hill-Climbing

### a. Deskripsi

Steepest Ascent Hill-Climbing merupakan varian Hill-Climbing yang selalu memilih neighbor terbaik di setiap iterasi. Algortima ini memeriksa seluruh neighbor dari state sekarang terlebih dahulu, kemudian berpindah ke neighbor dengan nilai objective function yang lebih tinggi. Proses ini diulang terus selama masih ada neighbor yang lebih baik. Jika sudah tidak ada neighbor yang lebih baik, pencarian berhenti karena state sekarang sudah mencapai local optimum.

Pada persoalan ini, algoritma berusaha mencari susuna kendaraan dengan total ShippingFee setinggi mungkin. Pencarian ini dibatasi maksimal 500 iterasi.

### b. Source Code

steepestascent_hc dari file src/hill_climbing.py

```
def steepestascent_hc(problem, initial_state, get_neighbors,
                      objective="fee", max_iterations=500):

    if not cek_placement(problem, initial_state):
        raise ValueError("Initial state tidak valid")

    current_state = initial_state
    current_score = nilai_state(problem, current_state, objective)

    history = [current_score]
    iterations = 0

    while iterations < max_iterations:
        best_state = None
        best_score = current_score

        for neighbor in get_neighbors(problem, current_state):
            if not cek_placement(problem, neighbor):
                continue
```

```
            neighbor_score = nilai_state(problem, neighbor,
objective)

            if neighbor_score > best_score:
                best_state = neighbor
                best_score = neighbor_score

        if best_state is None:
            break

        current_state = best_state
        current_score = best_score

        iterations += 1
        history.append(current_score)

    return {
        "final_state": current_state,
        "final_score": current_score,
        "history": history,
        "iterations": iterations
    }
```

**c. Penjelasan Source Code**

- **Fungsi nilai_state**

  Fungsi nilai_state digunakan untuk menghitung nilai suatu state. Fungsi ini memanggil value_fee jika objective function yang dipilih adalah ShippingFee, atau value_eta jika yang dipilih adalah ETA.

- **Fungsi steepestascent_hc**

  Fungsi ini menerima data persoalan, initial state, fungsi pembuat neighbor, objective function, dan batas iterasi. Pada setiap itrasim seluruh neighbor diambil dari all_neighbors, lalu dicari neighbor dengan nilai tertinggi yang lebih besar dari nilai sekarang. Jika ditemukan, state sekarang diganti dengan neighbor tersebut dan nilainya disimpan ke history. Jika tidak ditemukan, iterasi berhenti. Fungsi ini mengembalikan state akhir, nilai akhir, riwayat nilai, dan jumlah iterasi.

**d. Hasil Eksperimen**

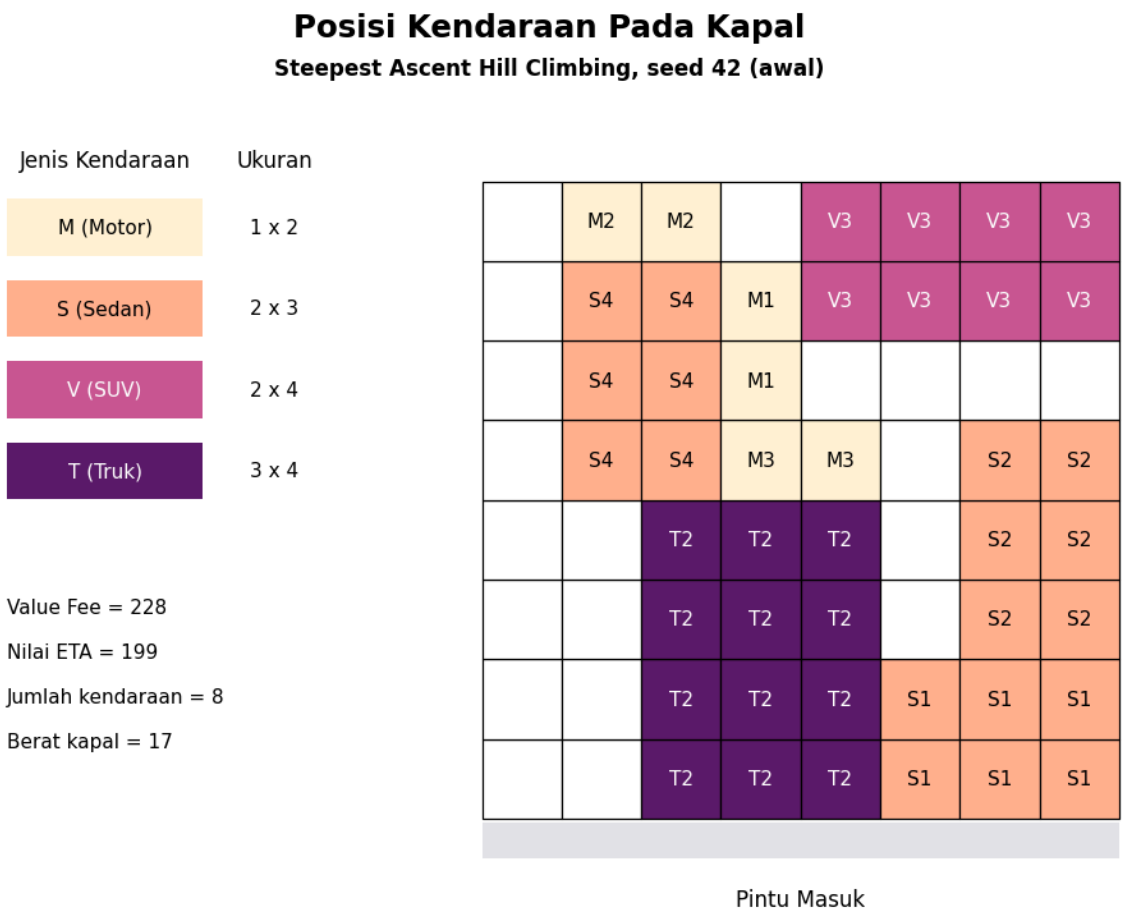
**Tabel 2.4 Hasil Steepest Ascent Hill-Climbing**

| Percobaan | Value Awal | Value Akhir | Iterasi | Durasi (detik) |
|---|---|---|---|---|
| 1 | 228 | 284 | 3 | 0,053 |

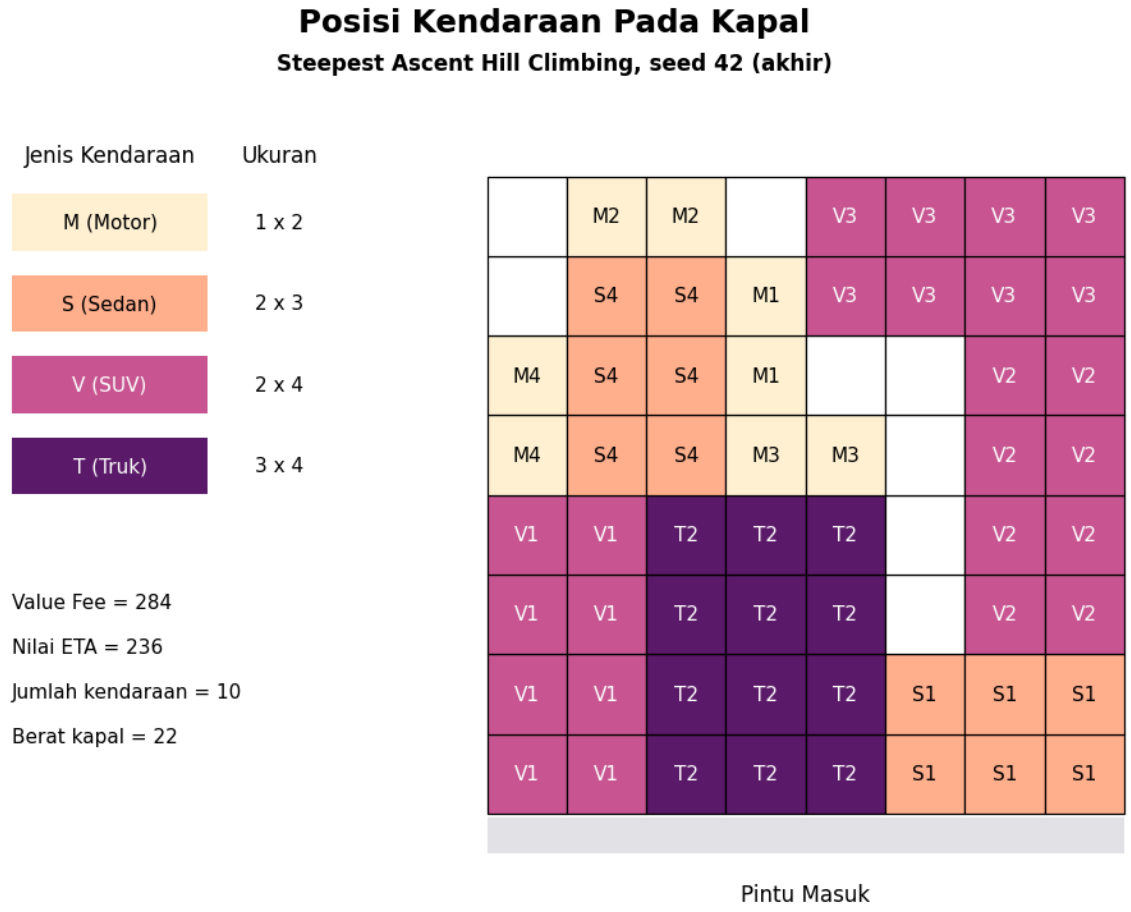
| | | | | |
|---|---|---|---|---|
| 2 | 228 | 254 | 3 | 0,054 |
| 3 | 178 | 284 | 6 | 0,087 |

Percobaan Steepest Ascent dengan seeds 42

**Gambar 2.1** Visualisasi awal Steepest Ascent Hill-Climbing



**Gambar 2.2** Visualisasi akhir Steepest Ascent Hill-Climbing




**Gambar 2.3** Grafik Steepest Ascent Hill-Climbing



**e. Analisis**

Steepest Ascent Hill-Climbing berhenti sangat cepat, yaitu hanya 3 hingga 6 iterasi dengan durasi kurang dari 0,1 detik. Setiap iterasi langsung menambil perubahan dengan kenaikan nilai paling besar, sehingga grafik naik tajam di awal lalu berhenti. Rata-rata nilai akhirnya adalah 274 atau sekitar 89% dari nilai optimum 308.

Ketiga percobaan berhenti di local optimum. Pada percobaan 1 dan 3, kapal sudah terisi penuh sampai kapasitasnya. Untuk mendapatkan nilai yang lebih tinggi, salah satu kendaraan harus dikeluarkan lalu diganti dengan kombinasi lain. Namun, langkah mengeluarkan kendaraan akan mengurangi value, sehingga tidak pernah dipilih oleh algoritma ini. Pada percobaan 2, masih ada sisa kapasitas, tetapi sudah tidak ada ruang kosong yang muat untuk kendaraan lain. Hasil ini juga menunjukkan bahwa nilai akhir sangat bergantung pada initial state. Percobaan 1 dan 2 memiliki value awal yang sama, tetapi berakhir di value yang berbeda karena susunan awalnya berbeda.

## 2.5 Simulated Annealing

**a. Deskripsi**

*Simulated Annealing* adalah algoritma *local search* yang terinspirasi dari proses pendinginan logam. Algoritma ini memperbolehkan berpindah ke neighbor yang lebih buruk dengan peluang tertentu, tidak seperti algoritma pada *Hill-Climbing*. Peluang tersebut bernilai besar ketika suhunya masih tinggi dan akan semakin kecil ketika suhunya turun. Dengan cara ini, algoritma dapat keluar dari *local optimum* di awal pencarian, lalu menjadi semakin stabil di akhir pencarian.

Proses *Simulated Annealing* adalah sebagai berikut.

1. Membuat initial state secara acak (*random*).
2. Mengambil satu neighbor acak dari state yang sekarang.
3. Mengecek neighbor.
   - Jika neighbor lebih baik, algoritma langsung berpindah ke neighbor tersebut.
   - Jika neighbor lebih buruk, algoritma berpindah dengan peluang e pangkat selisih nilai dibagi suhu ( $e^{\Delta E/T}$ )
4. Menurunkan suhu dengan mengalikan dengan alpha di setiap iterasi.
5. Mengulangi langkah 2 sampai 5 hingga suhunya mencapai suhu minimal (T_min).

**b. Source Code**

import library

```
import math
```

```python
import random
import time

from core import value_fee, value_eta, cek_placement
from neighbor import random_state, random_neighbor, all_neighbors
```

calculate_suhu dari file src/sa.py

```python
def calculate_suhu(t, T0, alpha, T_min):
    T = T0 * (alpha ** t)

    if T < T_min:
        return 0

    return T
```

cek_localoptimum dari file src/sa.py

```python
def cek_localoptimum(problem, state, objective="fee"):
    score_function = value_eta if objective == "eta" else value_fee

    v = score_function(problem, state)

    for nb in all_neighbors(problem, state):
        if score_function(problem, nb) > v:
            return False

    return True
```

simulated_annealing dari file src/sa.py

```python
def simulated_annealing(
    problem,
    T0=100.0,
    alpha=0.9998,
    T_min=0.1,
    initial=None,
    seed=None,
    objective="fee"
):
    if objective not in ("fee", "eta"):
        raise ValueError("Objective harus 'fee' atau 'eta'")
    if T0 <= 0:
        raise ValueError("T0 harus lebih besar dari 0")
    if not 0 < alpha < 1:
        raise ValueError("Alpha harus berada di antara 0 dan 1")
```

```
    if T_min <= 0:
        raise ValueError("T_min harus lebih besar dari 0")
    if seed is not None:
        random.seed(seed)

    score_function = value_eta if objective == "eta" else value_fee
    start = time.time()
    current = initial if initial is not None else
random_state(problem)

    if not cek_placement(problem, current):
        raise ValueError("Initial state tidak valid")

    cur_val = score_function(problem, current)
    best, best_val = current, cur_val
    awal = current
    history = [cur_val]
    probs = []
    stuck = 0
    worse_accepted = 0
    t = 0

    while True:
        t += 1
        T = calculate_suhu(t, T0, alpha, T_min)

        if T == 0:
            break

        nxt = random_neighbor(problem, current)

        if nxt is None:
            stuck += 1
            probs.append(None)
            history.append(cur_val)
            continue

        dE = score_function(problem, nxt) - cur_val

        if dE > 0:
            current, cur_val = nxt, cur_val + dE
            probs.append(None)
        else:
            p = math.exp(dE / T)
```

```
            probs.append(p)

            if random.random() < p:
                current, cur_val = nxt, cur_val + dE
                if dE < 0:
                    worse_accepted += 1
            else:
                stuck += 1
        if cur_val > best_val:
            best, best_val = current, cur_val

        history.append(cur_val)

    final_local_opt = cek_localoptimum(problem, current, objective)

    return {
        "algo": "SA",
        "params": {
            "T0": T0,
            "alpha": alpha,
            "T_min": T_min,
            "seed": seed,
            "objective": objective
        },
        "initial_state": awal,
        "final_state": current,
        "final_value": cur_val,
        "best_state": best,
        "best_value": best_val,
        "iterations": t - 1,
        "duration": time.time() - start,
        "history": history,
        "probs": probs,
        "stuck": stuck,
        "stuck_freq": stuck / max(1, t - 1),
        "worse_accepted": worse_accepted,
        "final_local_opt": final_local_opt
    }
```

**c. Penjelasan Source Code**

- **Fungsi calculate_suhu**

  Fungsi `calculate_suhu` digunakan untuk menghitung suhu pada iterasi tertentu. Jika suhunya sudah di bawah suhu minimal, fungsi ini mengembalikan nilai 0 sebagai tanda bahwa pencariannya harus berhenti.

- **Fungsi simulated_annealing**

  Fungsi `simulated_annealing` akan menjalankan algoritma *Simulated Annealing*. Pada setiap iterasi, fungsi ini mengambil satu neighbor menggunakan `random_neighbor` lalu menghitung selisih nilainya dengan state sekarang. Neighbor yang lebih baik akan langsung diterima, sedangkan neighbor yang lebih buruk akan diterima berdasarkan peluang. Nilai peluang tiap iterasi ini akan disimpan untuk membuat grafik. Fungsi ini juga mencatat jumlah iterasi yang tidak mengubah state sebagai kondisi *stuck.*
- **Fungsi cek_localoptimum**

  Fungsi `cek_localoptimum` digunakan untuk memeriksa apakah state akhir sudah merupakan local optimum, yaitu tidak ada lagi neighbor yang nilainya lebih tinggi dibanding state sekarang.

**d. Hasil Eksperimen**

**Tabel 2.5 Hasil Simulated Annealing**

| Percobaan | Value Awal | Value Akhir | Value Terbaik | Durasi | Frekuensi *stuck* |
|---|---|---|---|---|---|
| 1 | 188 | 296 | 308 | 2,24 | 0,42 |
| 2 | 196 | 296 | 308 | 2,23 | 0,41 |
| 3 | 248 | 296 | 306 | 2,87 | 0,36 |

Percobaan Simulated Annealing

**Gambar 2.4** State awal dan state akhir percobaan 1 yang mencapai nilai 296

**Posisi Kendaraan Pada Kapal**

**Simulated Annealing kasus 1, run 1 (awal)**

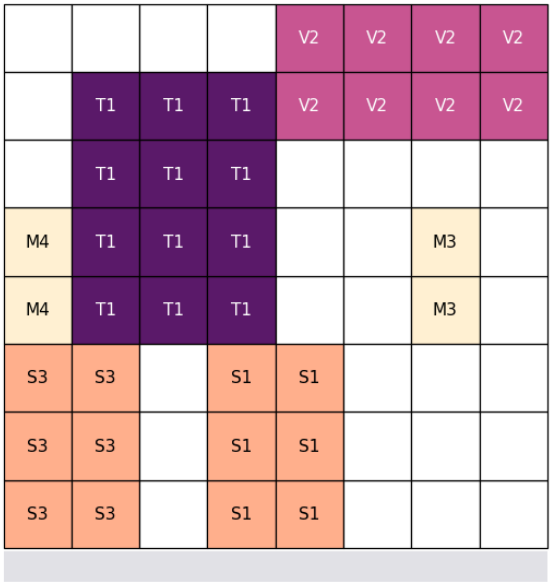
| Jenis Kendaraan | Ukuran |
|---|---|
| M (Motor) | 1 x 2 |
| S (Sedan) | 2 x 3 |
| V (SUV) | 2 x 4 |
| T (Truk) | 3 x 4 |

Value Fee = 188

Nilai ETA = 134

Jumlah kendaraan = 6

Berat kapal = 14




Pintu Masuk

**Posisi Kendaraan Pada Kapal**

**Simulated Annealing kasus 1, run 1 (akhir)**

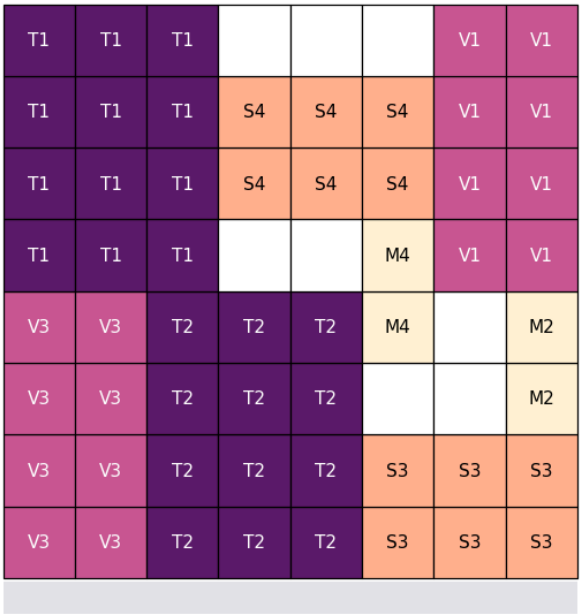
| Jenis Kendaraan | Ukuran |
|---|---|
| M (Motor) | 1 x 2 |
| S (Sedan) | 2 x 3 |
| V (SUV) | 2 x 4 |
| T (Truk) | 3 x 4 |

Value Fee = 296

Nilai ETA = 261

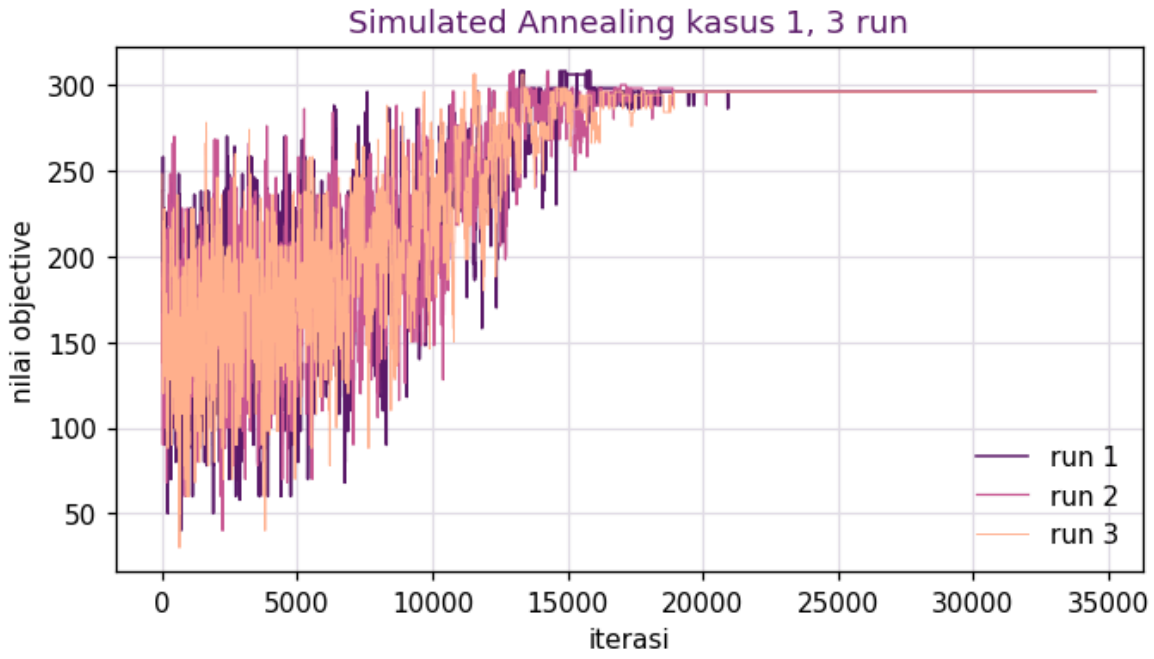
Jumlah kendaraan = 8

Berat kapal = 22

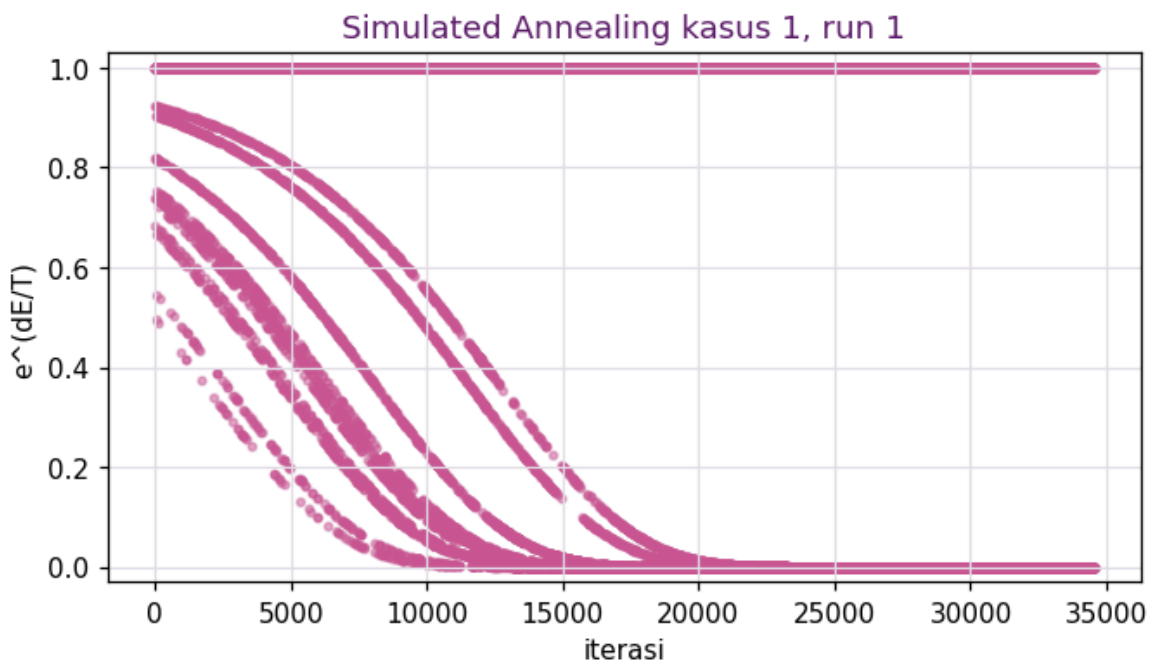



Pintu Masuk

**Gambar 2.5** Grafik nilai Objective Function ketiga percobaan




**Gambar 2.6** Grafik peluang terhadap iterasi percobaan 1




**e. Analisis**

*Simulated Annealing* menghasilkan nilai akhir yang sama pada ketiga percobaan, yaitu 296 atau sekitar 96% dari nilai optimum. Hasil ini tetap sama walaupun nilai awalnya berbeda, sehingga Simulated Annealing tidak terlalu bergantung pada initial state. Di awal pencarian, suhunya masih tinggi sehingga algoritma sering menerima neighbor yang lebih buruk dan grafiknya naik turun.

Langkah ini membuat algoritma dapat keluar dari susunan yang sudah penuh dan mencoba kombinasi kendaraan lain.

Seiring turunnya suhu, peluang menerima neighbor yang lebih buruk semakin kecil hingga mendekati nol. Pada akhir pencarian, algoritma menjadi mirip *Hill-Climbing*. Frekuensi *stuck* sekitar 0,4 menunjukkan bahwa kira-kira 4 dari 10 iterasi tidak mengubah state.

Selama proses berjalan, *Simulated Annealing* sempat mencapai nilai optimum 308 pada percobaan 1 dan 2. Akan tetapi, state tersebut kemudian ditinggalkan karena algoritma menerima neighbor yang lebih buruk, dan hasil yang dikembalikan adalah state terakhir dengan nilai 296. Dibanding yang lain, *Simulated Annealing* adalah algoritma paling lama karena jumlah iterasinya sangat banyak.

## 2.6 Genetic Algorithm

### a. Deskripsi

Genetic Algorithm merupakan algoritma local search berbasis populasi yang meniru proses evolusi. Setiap state disebut individu, dan posisi setiap kendaraan di dalamnya disebut gen. Individu dengan nilai lebih tinggi memiliki peluang lebih besar untuk dipilih menjadi induk. Dua induk kemudian digabungkan melalui crossover dan sesekali diubah melalui mutasi untuk menghasilkan populasi baru. Satu kali pembentukan populasi baru dihitung sebagai satu iterasi. Proses Genetic Algorithm pada persoalan ini adalah sebagai berikut.

1. Membuat populasi awal berisi state acak yang valid.
2. Menghitung nilai fitness setiap individu menggunakan total ShippingFee.
3. Memilih dua induk dengan metode roulette wheel, yaitu individu dengan fitness lebih tinggi lebih mungkin terpilih.
4. Melakukan crossover dengan peluang 0,9 menggunakan satu titik potong acak.
5. Memperbaiki anak yang tidak valid menggunakan fungsi fix_anak.
6. Melakukan mutasi dengan peluang 0,3 menggunakan satu langkah swap, move, atau rotate.
7. Menyalin satu individu terbaik ke populasi berikutnya, sehingga nilai terbaik tidak pernah turun.
8. Mengulangi langkah 2 sampai 7 sebanyak jumlah iterasi.

Anak hasil crossover bisa saja tidak valid karena kendaraan dari dua induk menempati sel yang sama atau beratnya melebihi kapasitas. Oleh karena itu, kami membuat heuristik perbaikan pada fungsi fix_anak. Kendaraan dimasukkan kembali satu per satu dengan urutan acak. Posisi dari anak dicoba terlebih dahulu, kemudian posisi kendaraan yang sama dari induk lainnya. Jika

keduanya gagal, kendaraan dikeluarkan dari kapal. Heuristik ini masih termasuk local search karena hanya menggunakan posisi yang sudah dimiliki oleh salah satu induk.

Peluang crossover 0,9 dan mutasi 0,3 dipilih dari uji coba awal beberapa kombinasi peluang. Hasil semua kombinasi tidak jauh berbeda, sehingga kami memilih nilai yang membuat crossover menjadi cara utama membentuk anak dan mutasi sebagai tambahan variasi.

**b. Source Code**

import library

```
import random
from core import OUT, cek_placement, value_fee, value_eta
from neighbor import random_state, random_neighbor
```

count_nilai_fitness dari file src/geneticalgorithm.py

```
def count_nilai_fitness(problem, state, objective):
    if objective == "eta":
        return value_eta(problem, state)
    return value_fee(problem, state)
```

choose_induk dari file src/geneticalgorithm.py

```
def choose_induk(populasi, fitness):
    return random.choices(populasi, weights=fitness)[0]
```

crossover dari file src/geneticalgorithm.py

```
def crossover(a, b):
    titik = random.randint(1, len(a) - 1)
    return a[:titik] + b[titik:], b[:titik] + a[titik:]
```

fix_anak dari file src/geneticalgorithm.py

```
def fix_anak(problem, anak, cadangan):
    hasil = [OUT] * len(anak)
    urutan = list(range(len(anak)))
    random.shuffle(urutan)
    for i in urutan:
        for posisi in [anak[i], cadangan[i]]:
            if posisi == OUT:
```

```
                continue
            hasil[i] = posisi

            if cek_placement(problem, tuple(hasil)):
                break
            hasil[i] = OUT
    return tuple(hasil)
```

mutation dari file src/geneticalgorithm.py

```
def mutation(problem, state):
    baru = random_neighbor(problem, state)
    if baru is None:
        return state
    return baru
```

genetic_algorithm dari file src/geneticalgorithm.py

```
def genetic_algorithm(problem, objective="fee", jumlah_populasi=50,
iterasi=200):
    populasi = []
    for i in range(jumlah_populasi):
        populasi.append(random_state(problem))

    fitness = [count_nilai_fitness(problem, s, objective) for s in
populasi]

    terbaik = populasi[fitness.index(max(fitness))]
    awal = terbaik
    riwayat_max = [max(fitness)]
    riwayat_rata = [sum(fitness) / jumlah_populasi]

    for gen in range(iterasi):
        baru = [populasi[fitness.index(max(fitness))]]

        while len(baru) < jumlah_populasi:
            a = choose_induk(populasi, fitness)
            b = choose_induk(populasi, fitness)
            if random.random() < 0.9:
                anak1, anak2 = crossover(a, b)
            else:
                anak1, anak2 = a, b

            if not cek_placement(problem, anak1):
               anak1 = fix_anak(problem, anak1, b)
```

```
            if not cek_placement(problem, anak2):
                anak2 = fix_anak(problem, anak2, a)

            for anak in [anak1, anak2]:
                if random.random() < 0.3:
                    anak = mutation(problem, anak)
                if len(baru) < jumlah_populasi:
                    baru.append(anak)

        populasi = baru
        fitness = [count_nilai_fitness(problem, s, objective) for s in
populasi]

        if max(fitness) > count_nilai_fitness(problem, terbaik,
objective):
            terbaik = populasi[fitness.index(max(fitness))]

        riwayat_max.append(max(fitness))
        riwayat_rata.append(sum(fitness) / jumlah_populasi)

    return awal, terbaik, riwayat_max, riwayat_rata
```

**c. Penjelasan Source Code**

- **Fungsi count_nilai_fitness**
  Fungsi count_nilai_fitness digunakan untuk menghitung nilai fitness suatu individu menggunakan value_fee atau value_eta.
- **Fungsi choose_induk**
  Fungsi choose_induk digunakan untuk memilih satu induk dengan metode roulette wheel berdasarkan nilai fitness.
- **Fungsi crossover**
  Fungsi crossover digunakan untuk menggabungkan dua induk dengan satu titik potong acak. Anak pertama mengambil bagian depan dari induk pertama dan bagian belakang dari induk kedua, sedangkan anak kedua sebaliknya.
- **Fungsi fix_anak**
  Fungsi fix_anak digunakan untuk memperbaiki anak yang melanggar batasan dengan heuristik yang telah dijelaskan sebelumnya.
- **Fungsi mutation**
  Fungsi mutation digunakan untuk mengubah satu bagian individu menggunakan random_neighbor.
- **Fungsi genetic_algorithm**
  Fungsi genetic_algorithm digunakan untuk menjalankan seluruh proses

Genetic Algorithm. Fungsi ini mengembalikan individu terbaik pada populasi awal sebagai state awal, individu terbaik selama proses sebagai state akhir, serta riwayat nilai maksimum dan rata-rata populasi di setiap iterasi.

**d. Hasil Eksperimen**

**Tabel 2.6 Hasil Genetic Algorithm percobaan A dengan populasi 50**

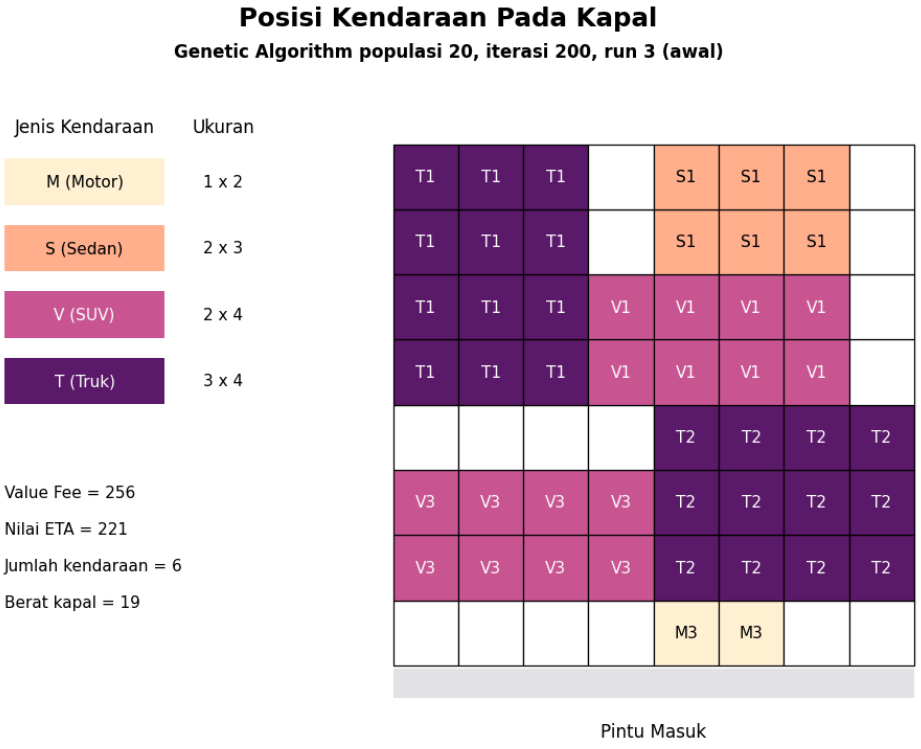
| Iterasi | Percobaan 1 | Percobaan 2 | Percobaan 3 | Rata-rata | Durasi rata-rata (detik) |
|---|---|---|---|---|---|
| 100 | 296 → 298 | 274 → 286 | 286 → 296 | 293,3 | 0,43 |
| 200 | 296 → 300 | 274 → 286 | 286 → 296 | 294,0 | 0,82 |
| 400 | 296 → 300 | 274 → 286 | 286 → 296 | 294,0 | 1,56 |

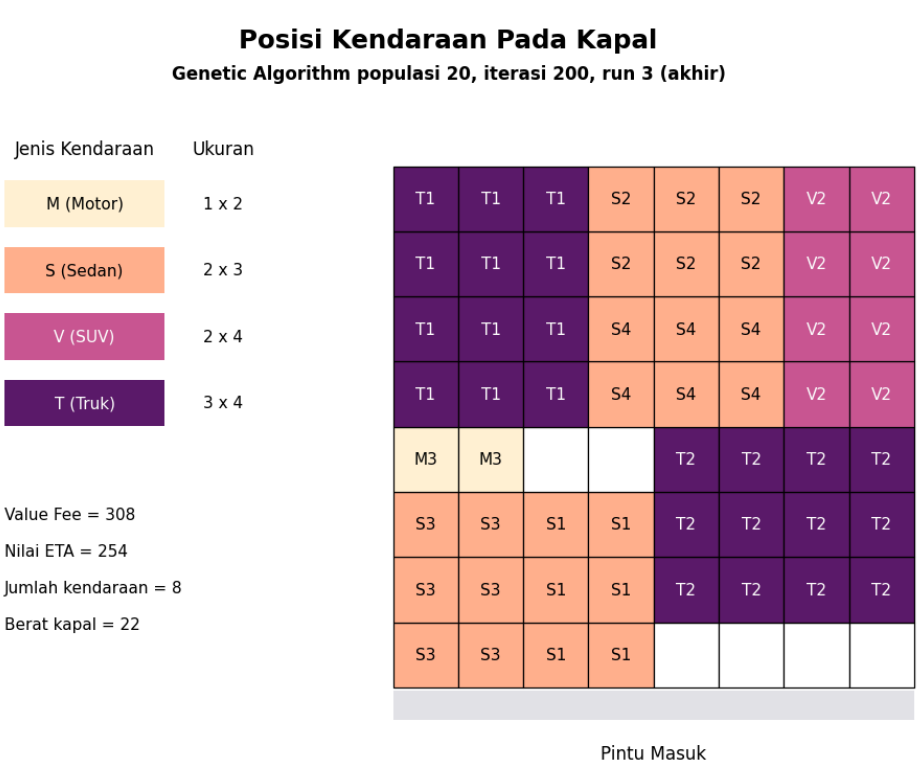
**Tabel 2.7 Hasil Genetic Algorithm percobaan B dengan iterasi 200**

| Populasi | Percobaan 1 | Percobaan 2 | Percobaan 3 | Rata-rata | Durasi rata-rata (detik) |
|---|---|---|---|---|---|
| 20 | 256 → 286 | 236 → 286 | 256 → 308 | 293,3 | 0,27 |
| 50 | 256 → 286 | 278 → 298 | 256 → 300 | 294,7 | 0,77 |
| 100 | 286 → 300 | 278 → 300 | 276 → 300 | 300,0 | 1,67 |

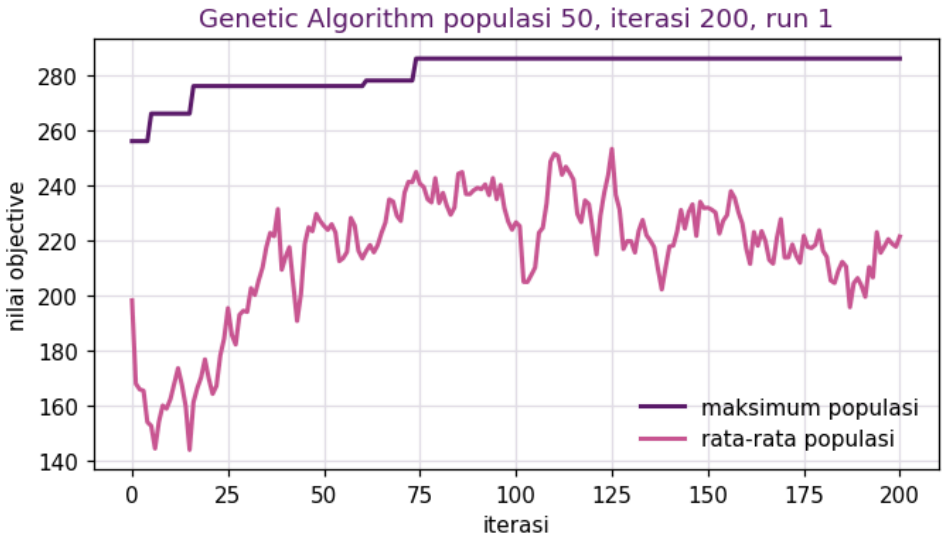
***Keterangan:***
*Tanda panah pada tabel menunjukkan perubahan dari nilai awal ke nilai akhir.*

**Gambar 2.7 State awal dan state akhir percobaan yang mencapai nilai 308,**



**Gambar 2.8 Grafik nilai maksimum dan rata-rata populasi,**




**e. Analisis**

Pada percobaan A, penambahan iterasi hampir tidak berpengaruh terhadap hasil. Sebagian besar percobaan sudah mencapai nilai akhirnya sebelum iterasi ke-100, dan hanya satu percobaan yang masih naik setelahnya. Rata-rata nilai hanya naik dari 293,3 menjadi 294,0, padahal durasinya naik hampir dua kali lipat setiap kali iterasi digandakan.

Pada percobaan B, penambahan populasi lebih berpengaruh. Populasi yang lebih besar memberikan nilai awal yang lebih baik karena lebih banyak state acak yang dicoba. Hasil akhirnya juga lebih stabil, terlihat dari populasi 100 yang menghasilkan nilai 300 pada ketiga percobaan. Populasi 20 paling tidak konsisten, tetapi salah satu percobaannya berhasil mencapai 308, yaitu nilai optimum kasus ini. Durasi juga naik hampir sebanding dengan jumlah populasi.

Grafik setiap percobaan menunjukkan nilai maksimum yang naik cepat di awal lalu mendatar. Rata-rata populasi tetap berada cukup jauh di bawah nilai maksimum. Hal ini terjadi karena perbaikan anak sering mengeluarkan kendaraan dari kapal, dan metode roulette wheel tidak terlalu membedakan individu yang nilainya berdekatan. Akibatnya, keragaman populasi tetap terjaga, tetapi peningkatan nilainya menjadi lebih lambat.

Secara rata-rata, Genetic Algorithm mencapai nilai 294,9 atau sekitar 96% dari nilai optimum. Semua hasil akhir sudah mengisi kapal sampai kapasitas penuh, dan perbedaannya hanya terletak pada kombinasi kendaraan yang dipilih.

## 2.7 Perbandingan Antaralgoritma

**Tabel 2.8 Perbandingan hasil ketiga algoritma**

| Algoritma | Rata-rata nilai akhir | Nilai terbaik | Persentase terhadap optimum | Durasi rata-rata (detik) |
|---|---|---|---|---|
| Steepest Ascent Hill-Climbing | 274,0 | 284 | 89% | 0,07 |
| Simulated Annealing | 296,0 | 296 | 96% | 2,45 |
| Genetic Algorithm | 294,9 | 308 | 96% | 0,92 |

Dari sisi kedekatan dengan global optimum, Genetic Algorithm merupakan satu-satunya algoritma yang menghasilkan nilai optimum 308. Simulated Annealing sempat mencapai nilai tersebut selama proses, tetapi hasil akhirnya adalah state terakhir dengan nilai 296. Steepest Ascent Hill-Climbing paling jauh dari optimum karena langsung berhenti di local optimum pertama yang ditemui. Simulated Annealing dan Genetic Algorithm lebih baik karena keduanya memiliki cara untuk keluar dari local optimum, yaitu melalui penerimaan neighbor yang lebih buruk dan melalui crossover serta mutasi.

Dari sisi hasil pencarian, rata-rata tertinggi diperoleh Simulated Annealing, kemudian Genetic Algorithm dengan selisih yang sangat kecil, lalu Steepest Ascent Hill-Climbing yang tertinggal sekitar 20 poin. Ketiga algoritma sudah mengisi kapal hingga penuh atau hampir penuh, sehingga perbedaan nilai berasal dari kombinasi kendaraan yang dipilih.

Dari sisi durasi, Steepest Ascent Hill-Climbing paling cepat karena hanya membutuhkan beberapa iterasi. Genetic Algorithm berada di urutan kedua dengan durasi yang bergantung pada jumlah populasi dan iterasi. Simulated Annealing paling lama karena menjalankan puluhan ribu iterasi. Meskipun begitu, satu iterasi Hill-Climbing memeriksa seluruh neighbor, sedangkan satu iterasi Simulated Annealing hanya memeriksa satu neighbor, sehingga jumlah iterasi keduanya tidak dapat dibandingkan secara langsung.

Dari sisi konsistensi, Simulated Annealing paling konsisten karena ketiga percobaannya menghasilkan nilai yang sama. Genetic Algorithm cukup konsisten, terutama pada populasi 100. Steepest Ascent Hill-Climbing paling tidak konsisten karena hasilnya sangat bergantung pada initial state.

## 2.8 Visualisasi dan Program Demo

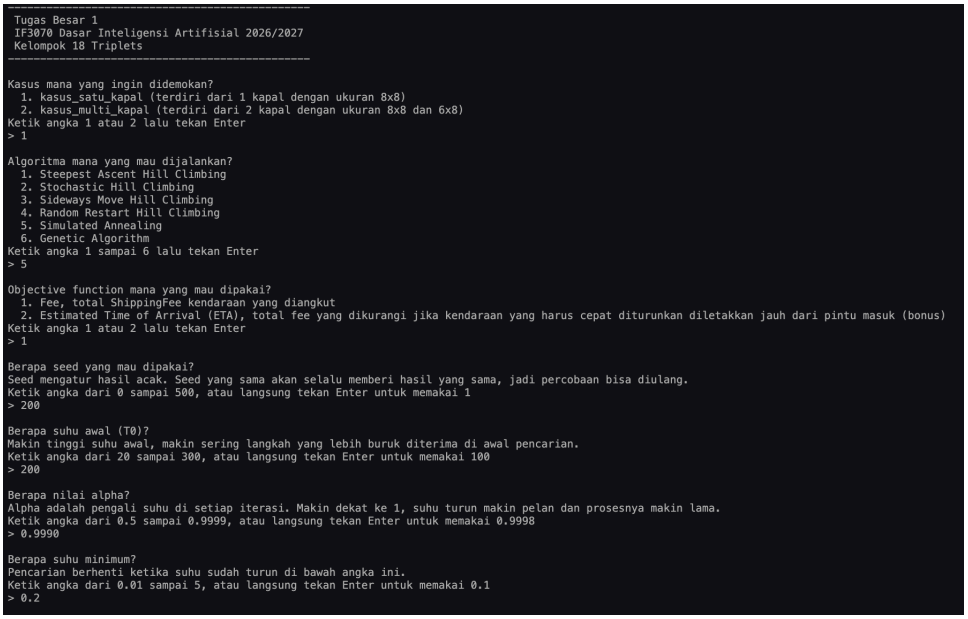
Visualisasi dibuat menggunakan library matplotlib dan digunakan oleh ketiga algoritma. Fungsi visual_kapal digunakan untuk menggambar satu state dalam bentuk grid geladak. Setiap kendaraan diberi warna sesuai jenisnya beserta label singkat seperti M1 untuk motor dan T1 untuk truk, sedangkan sisi depan kapal diberi tanda Pintu Masuk. Di samping grid ditampilkan legenda ukuran kendaraan, nilai ShippingFee, nilai ETA, jumlah kendaraan, dan berat muatan.

**Tabel 2.9 Fungsi visualisasi**

| Fungsi | Kegunaan |
|---|---|
| visual_kapal | Menggambar state awal dan state akhir |
| grafik_objective_kapal | Menggambar grafik nilai objective function terhadap iterasi untuk Hill-Climbing dan Simulated Annealing |
| visual_genetic_algorithm | Menggambar grafik nilai maksimum dan rata-rata populasi untuk Genetic Algorithm |
| grafik_prob | Menggambar grafik peluang terhadap iterasi untuk Simulated Annealing |
| visual_perbandingan | Menggambar beberapa grafik dalam satu gambar untuk perbandingan |
| grafik_restart | Menggambar grafik setiap restart pada Random Restart Hill-Climbing |

Program demo dibuat dalam bentuk menu di terminal pada file main.py. Pengguna memilih kasus, algoritma, objective function, seed, dan parameter algoritma. Setelah algoritma selesai, program menampilkan state awal dan akhir dalam bentuk teks, termasuk daftar kendaraan yang berada di luar kapal, serta ringkasan nilai, jumlah iterasi, dan durasi. Gambar state dan grafik juga disimpan secara otomatis ke folder output/hc, ouptut/sa, dan output/ga.

## Gambar 2.9 Tampilan menu awal



```
================================================
 Tugas Besar 1
 IF3070 Dasar Inteligensi Artifisial 2026/2027
 Kelompok 18 Triplets
================================================

Kasus mana yang ingin didemokan?
  1. kasus_satu_kapal (terdiri dari 1 kapal dengan ukuran 8x8)
  2. kasus_multi_kapal (terdiri dari 2 kapal dengan ukuran 8x8 dan 6x8)
Ketik angka 1 atau 2 lalu tekan Enter
> 1

Algoritma mana yang mau dijalankan?
  1. Steepest Ascent Hill Climbing
  2. Stochastic Hill Climbing
  3. Sideways Move Hill Climbing
  4. Random Restart Hill Climbing
  5. Simulated Annealing
  6. Genetic Algorithm
Ketik angka 1 sampai 6 lalu tekan Enter
> 5

Objective function mana yang mau dipakai?
  1. Fee, total ShippingFee kendaraan yang diangkut
  2. Estimated Time of Arrival (ETA), total fee yang dikurangi jika kendaraan yang harus cepat diturunkan diletakkan jauh dari pintu masuk (bonus)
Ketik angka 1 atau 2 lalu tekan Enter
> 1

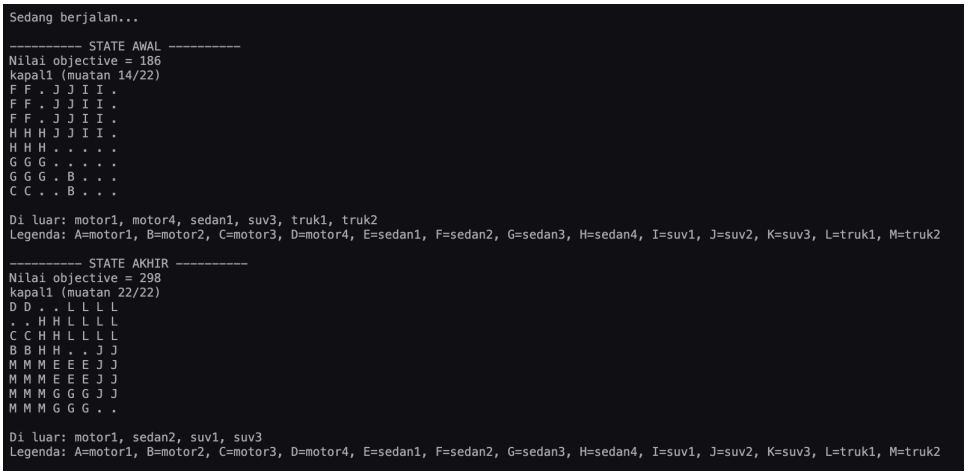
Berapa seed yang mau dipakai?
Seed mengatur hasil acak. Seed yang sama akan selalu memberi hasil yang sama, jadi percobaan bisa diulang.
Ketik angka dari 0 sampai 500, atau langsung tekan Enter untuk memakai 1
> 200

Berapa suhu awal (T0)?
Makin tinggi suhu awal, makin sering langkah yang lebih buruk diterima di awal pencarian.
Ketik angka dari 20 sampai 300, atau langsung tekan Enter untuk memakai 100
> 200

Berapa nilai alpha?
Alpha adalah pengali suhu di setiap iterasi. Makin dekat ke 1, suhu turun makin pelan dan prosesnya makin lama.
Ketik angka dari 0.5 sampai 0.9999, atau langsung tekan Enter untuk memakai 0.9998
> 0.9990

Berapa suhu minimum?
Pencarian berhenti ketika suhu sudah turun di bawah angka ini.
Ketik angka dari 0.01 sampai 5, atau langsung tekan Enter untuk memakai 0.1
> 0.2
```

## Gambar 2.10 Tampilan hasil di terminal dan pop up visualisasi



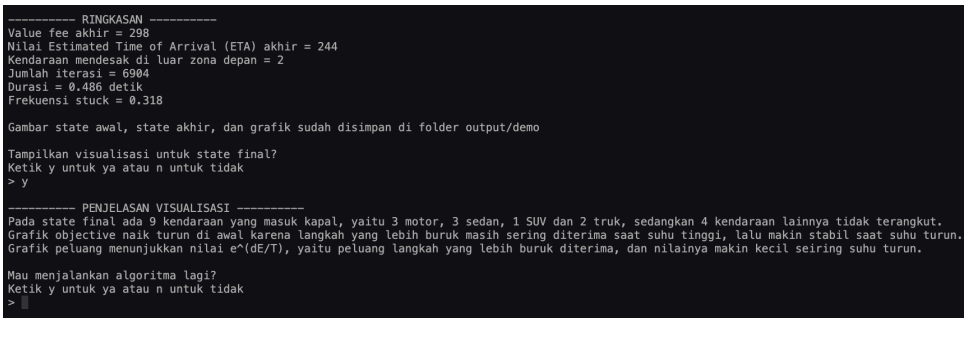
```
Sedang berjalan...

---------- STATE AWAL ----------
Nilai objective = 186
kapal1 (muatan 14/22)
F F . J J I I .
F F . J J I I .
F F . J J I I .
H H H J J I I .
H H H . . . . .
G G G . . . . .
G G G . B . . .
C C . . B . . .

Di luar: motor1, motor4, sedan1, suv3, truk1, truk2
Legenda: A=motor1, B=motor2, C=motor3, D=motor4, E=sedan1, F=sedan2, G=sedan3, H=sedan4, I=suv1, J=suv2, K=suv3, L=truk1, M=truk2

---------- STATE AKHIR ----------
Nilai objective = 298
kapal1 (muatan 22/22)
D D . . L L L L
. . H H L L L L
C C H H L L L L
B B H H . . J J
M M M E E E J J
M M M E E E J J
M M M G G G J J
M M M G G G . .

Di luar: motor1, sedan2, suv1, suv3
Legenda: A=motor1, B=motor2, C=motor3, D=motor4, E=sedan1, F=sedan2, G=sedan3, H=sedan4, I=suv1, J=suv2, K=suv3, L=truk1, M=truk2
```



```
---------- RINGKASAN ----------
Value fee akhir = 298
Nilai Estimated Time of Arrival (ETA) akhir = 244
Kendaraan mendesak di luar zona depan = 2
Jumlah iterasi = 6904
Durasi = 0.486 detik
Frekuensi stuck = 0.318

Gambar state awal, state akhir, dan grafik sudah disimpan di folder output/demo

Tampilkan visualisasi untuk state final?
Ketik y untuk ya atau n untuk tidak
> y

---------- PENJELASAN VISUALISASI ----------
Pada state final ada 9 kendaraan yang masuk kapal, yaitu 3 motor, 3 sedan, 1 SUV dan 2 truk, sedangkan 4 kendaraan lainnya tidak terangkut.
Grafik objective naik turun di awal karena langkah yang lebih buruk masih sering diterima saat suhu tinggi, lalu makin stabil saat suhu turun.
Grafik peluang menunjukkan nilai e^(dE/T), yaitu peluang langkah yang lebih buruk diterima, dan nilainya makin kecil seiring suhu turun.

Mau menjalankan algoritma lagi?
Ketik y untuk ya atau n untuk tidak
>
```

## Posisi Kendaraan Pada Kapal

**Simulated Annealing, seed 200 (akhir)**

| Jenis Kendaraan | Ukuran |
|---|---|
| M (Motor) | 1 x 2 |
| S (Sedan) | 2 x 3 |
| V (SUV) | 2 x 4 |
| T (Truk) | 3 x 4 |

Value Fee = 298

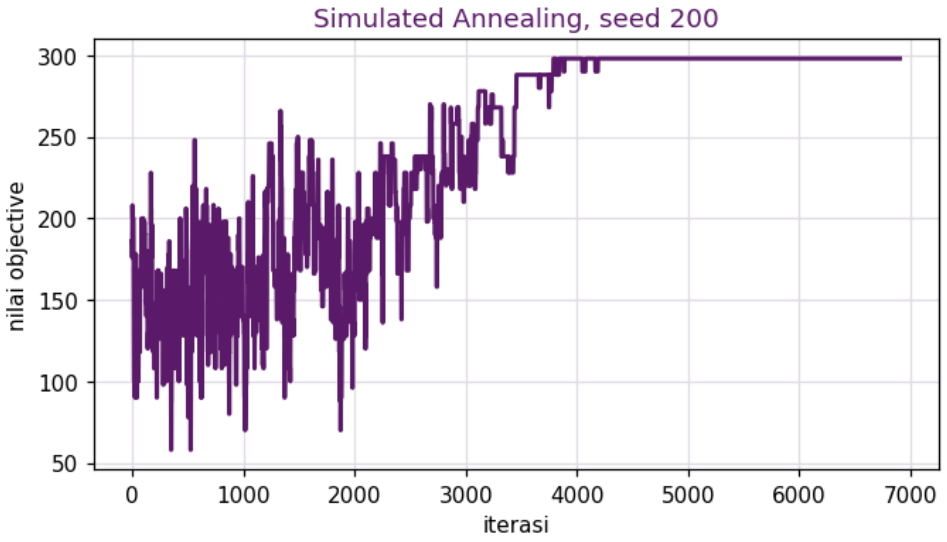
Nilai ETA = 244

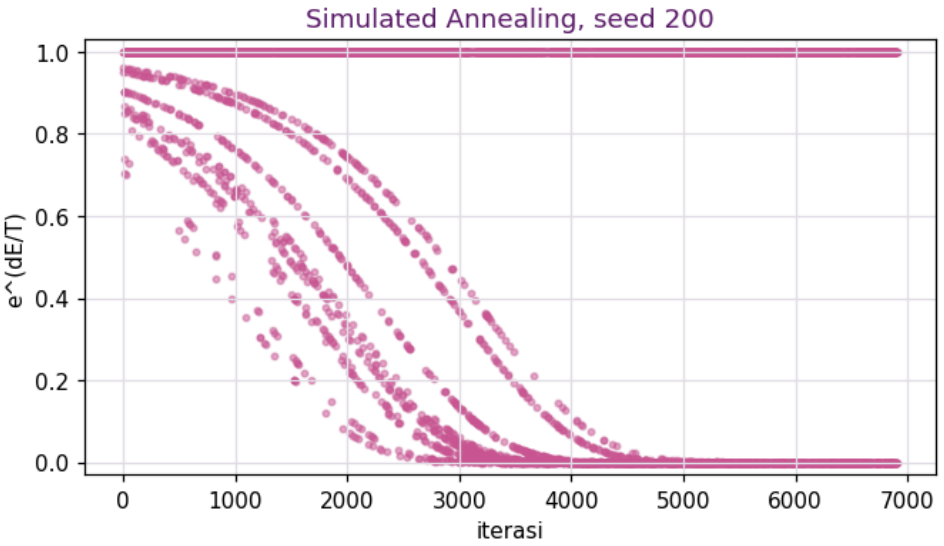
Jumlah kendaraan = 9

Berat kapal = 22

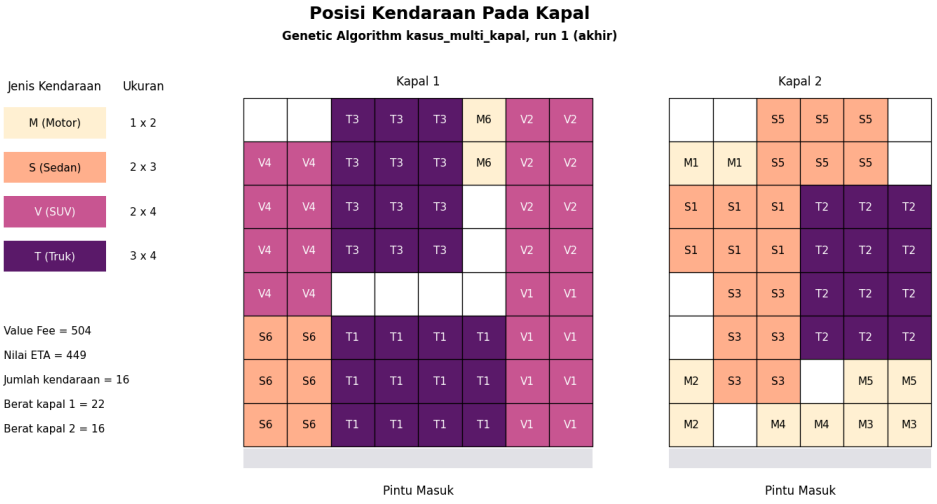
| | | | | | | | |
|---|---|---|---|---|---|---|---|
| T1 | T1 | T1 | V2 | V2 | V2 | V2 | |
| T1 | T1 | T1 | V2 | V2 | V2 | V2 | |
| T1 | T1 | T1 | | S1 | S1 | S3 | S3 |
| T1 | T1 | T1 | | S1 | S1 | S3 | S3 |
| | S4 | S4 | S4 | S1 | S1 | S3 | S3 |
| | S4 | S4 | S4 | T2 | T2 | T2 | T2 |
| M4 | | M3 | M2 | T2 | T2 | T2 | T2 |
| M4 | | M3 | M2 | T2 | T2 | T2 | T2 |

Pintu Masuk






**Gambar 2.11 Contoh visualisasi dua kapal**



# BAB III
# IMPLEMENTASI BONUS

## 3.1 Variasi Hill-Climbing

Ketiga varian berikut menggunakan initial state, seed, dan batas iterasi yang sama dengan Steepest Ascent Hill-Climbing pada subbab 2.4, sehingga hasilnya dapat dibandingkan secara langsung.

### 3.1.1 Stochastic Hill-Climbing

**a. Deskripsi**

Stochastic Hill-Climbing merupakan varian Hill-Climbing yang memilih neighbor secara acak. Algoritma ini tetap memeriksa seluruh neighbor, tetapi tidak selalu memilih yang terbaik. Dari semua neighbor yang lebih baik dari state sekarang, satu neighbor dipilih secara acak. Pencarian berhenti jika sudah tidak ada neighbor yang lebih baik.

**b. Source Code**

Stochastic_hc di file src/hill_climbing.py

```
def stochastic_hc(problem, initial_state, get_neighbors,
                            objective="fee", max_iterations=500):

    if not cek_placement(problem, initial_state):
        raise ValueError("Initial state tidak valid")

    current_state = initial_state
    current_score = nilai_state(problem, current_state, objective)

    history = [current_score]
    iterations = 0

    while iterations < max_iterations:
        better_neighbors = []

        for neighbor in get_neighbors(problem, current_state):
            if not cek_placement(problem, neighbor):
                continue

            neighbor_score = nilai_state(problem, neighbor, objective)

            if neighbor_score > current_score:
                better_neighbors.append((neighbor, neighbor_score))

        if not better_neighbors:
            break

        chosen_state, chosen_score = random.choice(better_neighbors)
```

```
        current_state = chosen_state
        current_score = chosen_score

        iterations += 1
        history.append(current_score)

    return {
        "final_state": current_state,
        "final_score": current_score,
        "history": history,
        "iterations": iterations
    }
```

**c. Penjelasan Source Code**

Fungsi stochastic_hc digunakan untuk menjalankan Stochastic Hill-Climbing. Fungsi ini mengumpulkan seluruh neighbor yang nilainya lebih tinggi ke dalam satu daftar, lalu memilih satu neighbor secara acak dari daftar tersebut. Bagian lainnya sama dengan fungsi steepestascent_hc.

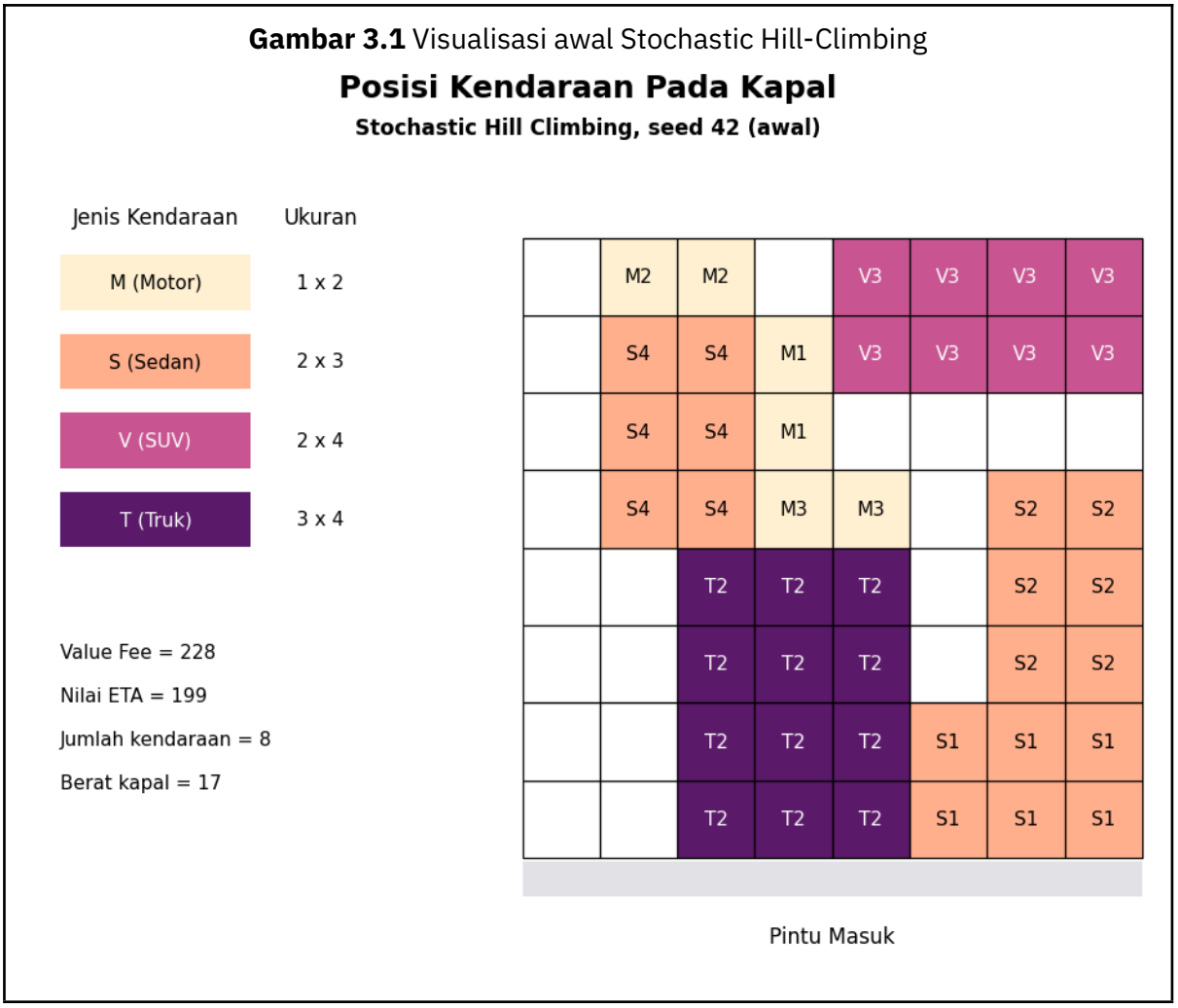
**d. Hasil Eksperimen**

**Tabel 3.1 Hasil Stochastic Hill-Climbing**

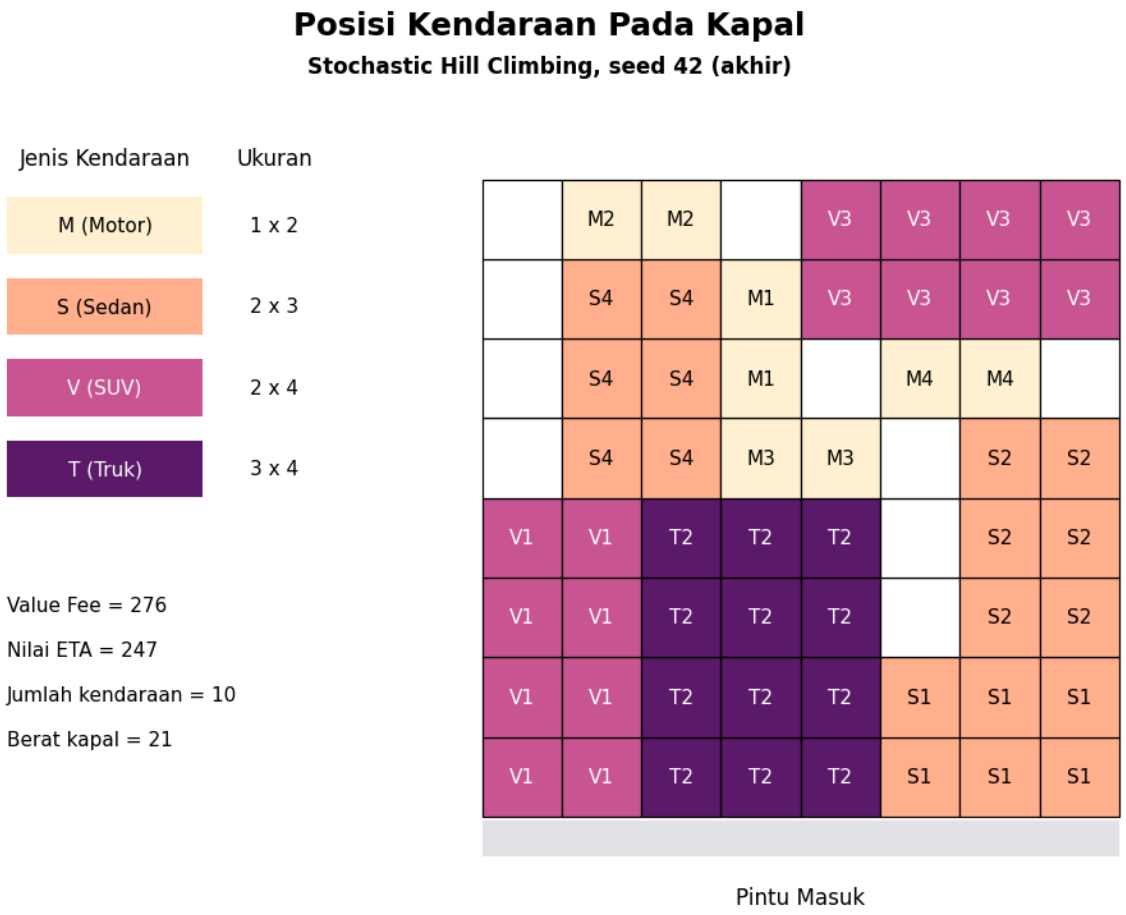
| Percobaan | Value Awal | Value Akhir | Iterasi | Durasi (detik) |
|---|---|---|---|---|
| 1 | 228 | 276 | 3 | 0,07 |
| 2 | 228 | 254 | 3 | 0,06 |
| 3 | 178 | 284 | 7 | 0,11 |

Percobaan Stochastic Hill Climbing dengan seeds 42

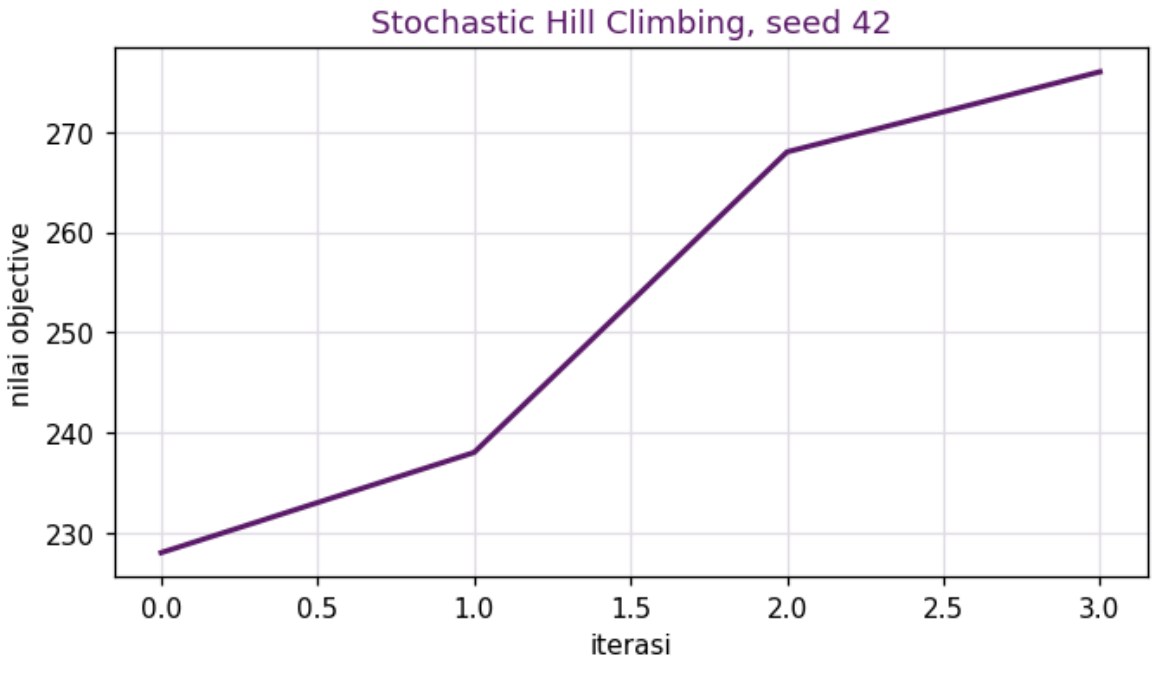
**Gambar 3.1** Visualisasi awal Stochastic Hill-Climbing



**Gambar 3.2** Visualisasi akhir Stochastic Hill-Climbing




**Gambar 3.3** Grafik Stochastic Hill-Climbing



### 3.1.2 Hill-Climbing with Sideways Move

**a. Deskripsi**

Hill-Climbing with Sideways Move merupakan varian Hill-Climbing yang boleh berpindah ke neighbor dengan nilai yang sama ketika tidak ada neighbor yang lebih baik. Perpindahan ini disebut sideways move dan bertujuan agar pencarian dapat melewati daerah yang datar. Jumlah sideways move berturut-turut dibatasi maksimal 20 kali, sesuai dengan spesifikasi.

**b. Source Code**

sideways_hc di file src/hill_climbing.py

```
def sideways_hc(problem, initial_state, get_neighbors,
                            objective="fee", max_iterations=500,
                            max_sideways=20):

    if not cek_placement(problem, initial_state):
        raise ValueError("Initial state tidak valid")

    current_state = initial_state
    current_score = nilai_state(problem, current_state,
objective)

    history = [current_score]
    iterations = 0
    sideways_count = 0
    total_sideways = 0

    visited = {tuple(current_state)}

    while iterations < max_iterations:
        best_state = None
        best_score = current_score
        equal_neighbors = []

        for neighbor in get_neighbors(problem, current_state):
            if not cek_placement(problem, neighbor):
                continue

            neighbor_score = nilai_state(problem, neighbor,
objective)

            if neighbor_score > best_score:
```

```
                best_state = neighbor
                best_score = neighbor_score

            elif neighbor_score == current_score:
                if tuple(neighbor) not in visited:
                    equal_neighbors.append(neighbor)

        if best_state is not None:
            current_state = best_state
            current_score = best_score
            sideways_count = 0
            visited = {tuple(current_state)}

        elif equal_neighbors and sideways_count < max_sideways:
            current_state = random.choice(equal_neighbors)
            sideways_count += 1
            total_sideways += 1
            visited.add(tuple(current_state))

        else:
            break

        iterations += 1
        history.append(current_score)

    return {
        "final_state": current_state,
        "final_score": current_score,
        "history": history,
        "iterations": iterations,
        "sideways_moves": total_sideways,
        "max_sideways": max_sideways
    }
```

**c. Penjelasan Source Code**

Fungsi sideways_hc digunakan untuk menjalankan Hill-Climbing with Sideways Move. Jika terdapat neighbor yang lebih baik, fungsi ini berpindah ke neighbor terbaik dan menghitung ulang sideways move dari nol. Jika tidak ada, fungsi ini memilih secara acak neighbor yang nilainya sama dan belum pernah dikunjungi, agar pencarian tidak bolak-balik ke state yang sama. Pencarian berhenti jika sideways move sudah mencapai batas atau tidak ada neighbor yang nilainya sama.

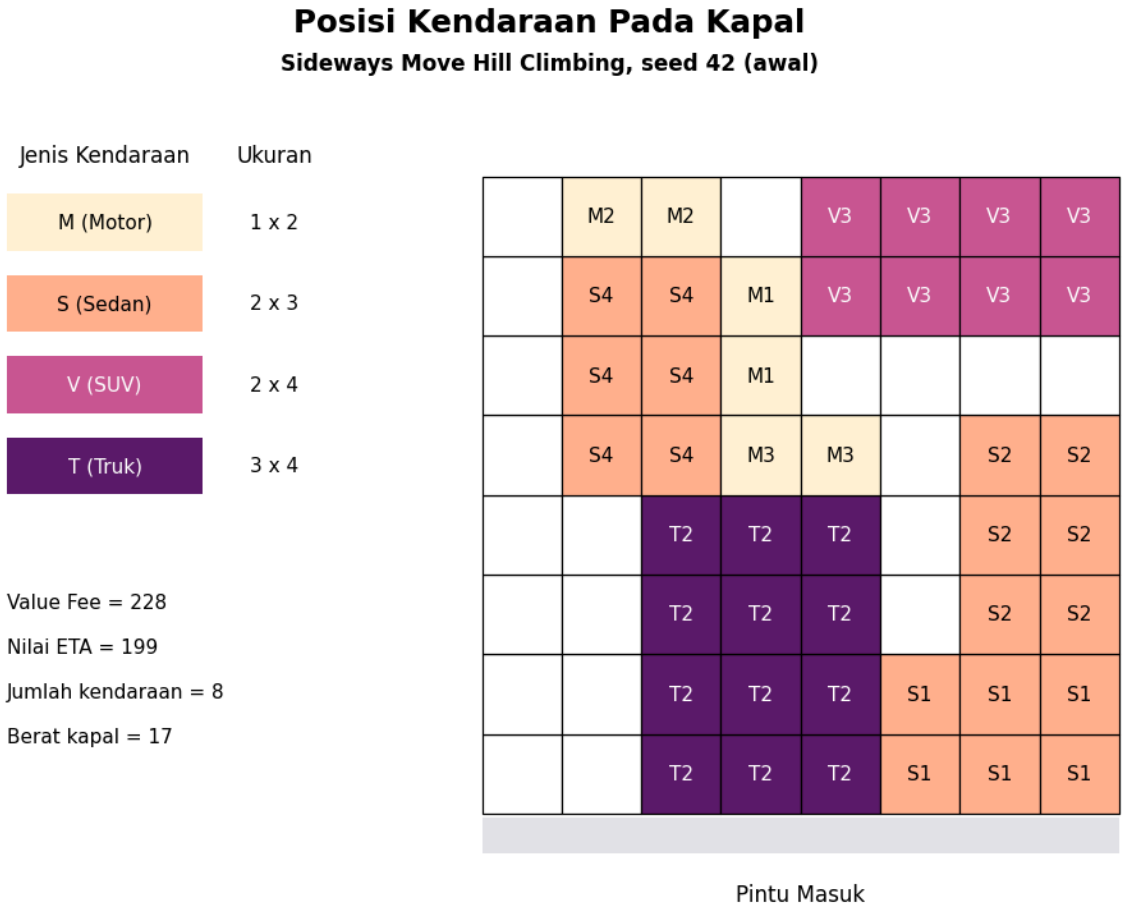
**d. Hasil Eksperimen**

**Tabel 3.2 Hasil Hill-Climbing with Sideways Move**

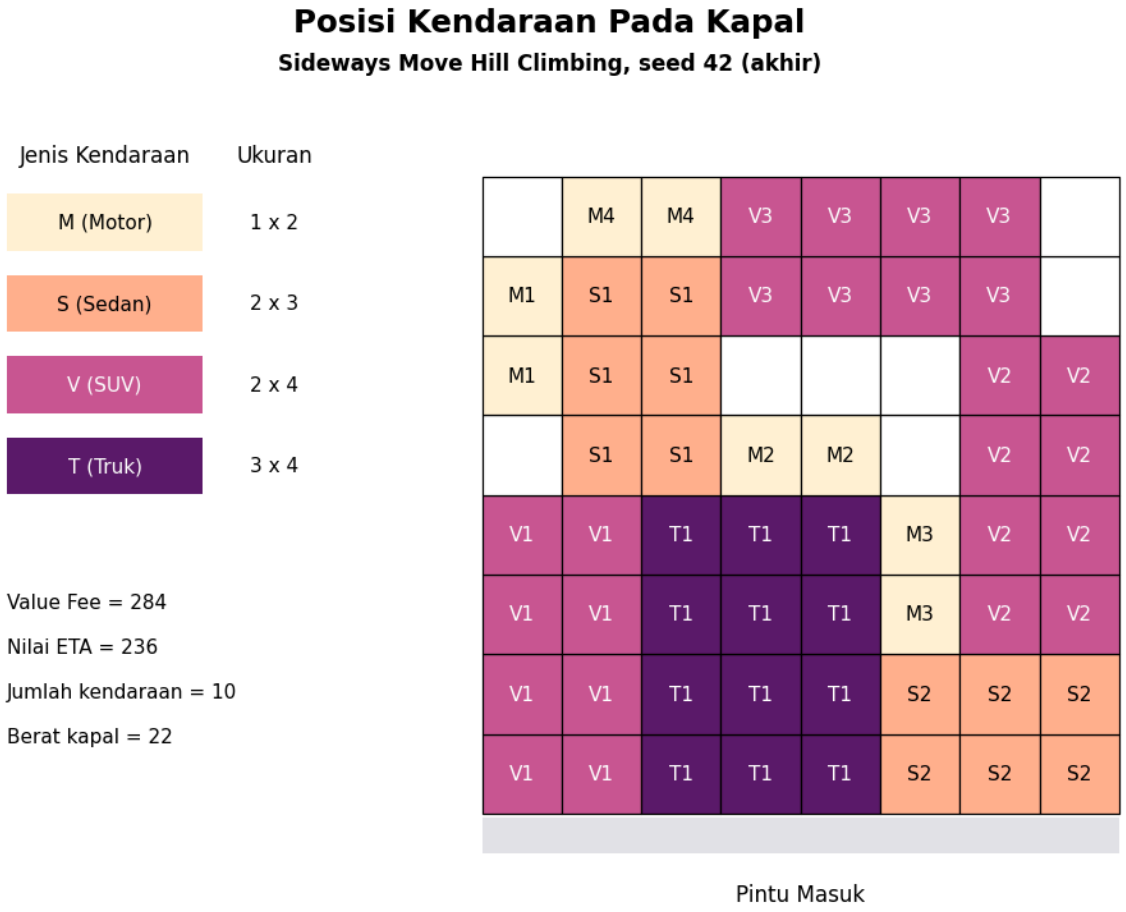
| Percobaan | Value Awal | Value Akhir | Iterasi | Jumlah sideways move |
|---|---|---|---|---|
| 1 | 228 | 284 | 23 | 20 |
| 2 | 228 | 286 | 29 | 25 |
| 3 | 178 | 284 | 26 | 20 |

Percobaan Sideways dengan seeds 42

**Gambar 3.4** Visualisasi awal Sideways Hill-Climbing



**Gambar 3.5** Visualisasi akhir Sideways Hill-Climbing




**Gambar 3.6** Grafik Sideways Hill-Climbing



### 3.1.3 Random Restart Hill-Climbing

**a. Deskripsi**

Random Restart Hill-Climbing merupakan varian Hill-Climbing yang menjalankan Steepest Ascent Hill-Climbing berkali-kali dari initial state yang berbeda, lalu mengambil hasil terbaik. Pencarian pertama menggunakan initial state yang sama dengan varian lain, sedangkan restart berikutnya menggunakan state acak yang baru. Jumlah restart dibatasi maksimal 10 kali, sesuai dengan spesifikasi.

**b. Source Code**

randomrestart_hc di file src/hill_climbing.py

```
def randomrestart_hc(problem, initial_state, get_neighbors,
                                random_state, objective="fee",
                                max_restarts=10, max_iterations=500):

    if not cek_placement(problem, initial_state):
        raise ValueError("Initial state tidak valid")

    best_result = None
    restart_scores = []
    restart_iterations = []
    all_histories = []

    total_iterations = 0

    for restart in range(max_restarts + 1):
        if restart == 0:
            start_state = initial_state
        else:
            start_state = random_state(problem)

        result = steepestascent_hc(
            problem,
            start_state,
            get_neighbors,
            objective=objective,
            max_iterations=max_iterations
        )

        restart_scores.append(result["final_score"])
        restart_iterations.append(result["iterations"])
        all_histories.append(result["history"])

        total_iterations += result["iterations"]
```

```
    if best_result is None:
        best_result = result
    elif result["final_score"] > best_result["final_score"]:
        best_result = result

return {
    "final_state": best_result["final_state"],
    "final_score": best_result["final_score"],
    "history": best_result["history"],
    "all_histories": all_histories,
    "iterations": total_iterations,
    "restarts": max_restarts,
    "restart_scores": restart_scores,
    "restart_iterations": restart_iterations,
    "max_restarts": max_restarts
}
```

**c. Penjelasan Source Code**

Fungsi randomrestart_hc digunakan untuk menjalankan Random Restart Hill-Climbing. Fungsi ini memanggil steepestascent_hc sebanyak satu kali pencarian awal ditambah 10 kali restart. Nilai akhir dan jumlah iterasi setiap restart disimpan, lalu hasil dengan nilai tertinggi dikembalikan sebagai solusi.

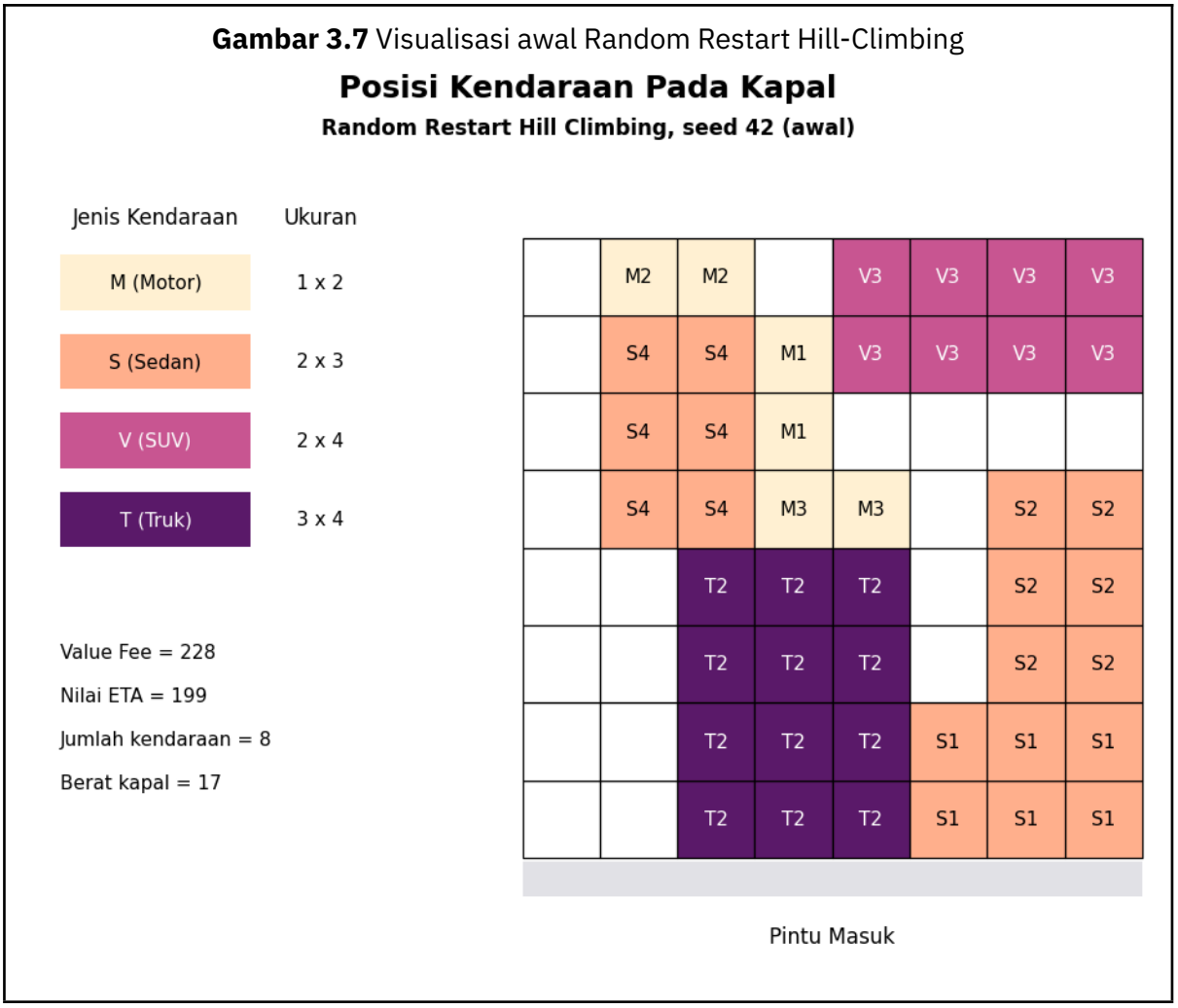
**d. Hasil Eksperimen**

**Tabel 3.3 Hasil Random Restart Hill-Climbing**

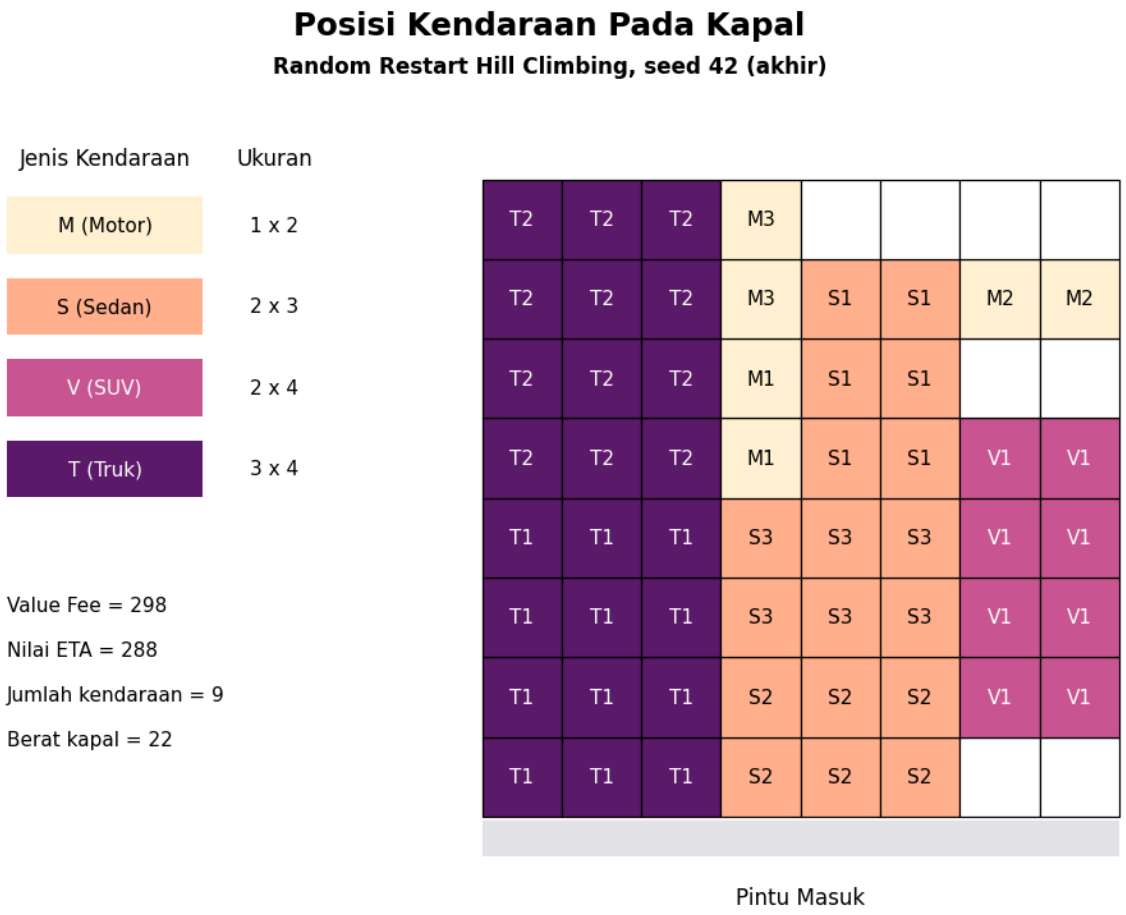
| Percobaan | Nilai akhir | Jumlah restart | Total iterasi | Iterasi per restart | Durasi (detik) |
|---|---|---|---|---|---|
| 1 | 298 | 10 | 39 | 3, 3, 5, 4, 5, 4, 6, 3, 2, 3, 1 | 0,76 |
| 2 | 298 | 10 | 36 | 3, 3, 3, 3, 4, 2, 6, 6, 2, 0, 4 | 0,66 |
| 3 | 300 | 10 | 46 | 6, 6, 2, 3, 5, 5, 2, 2, 6, 7, 2 | 0,78 |

Percobaan Random Restart dengan seeds 42

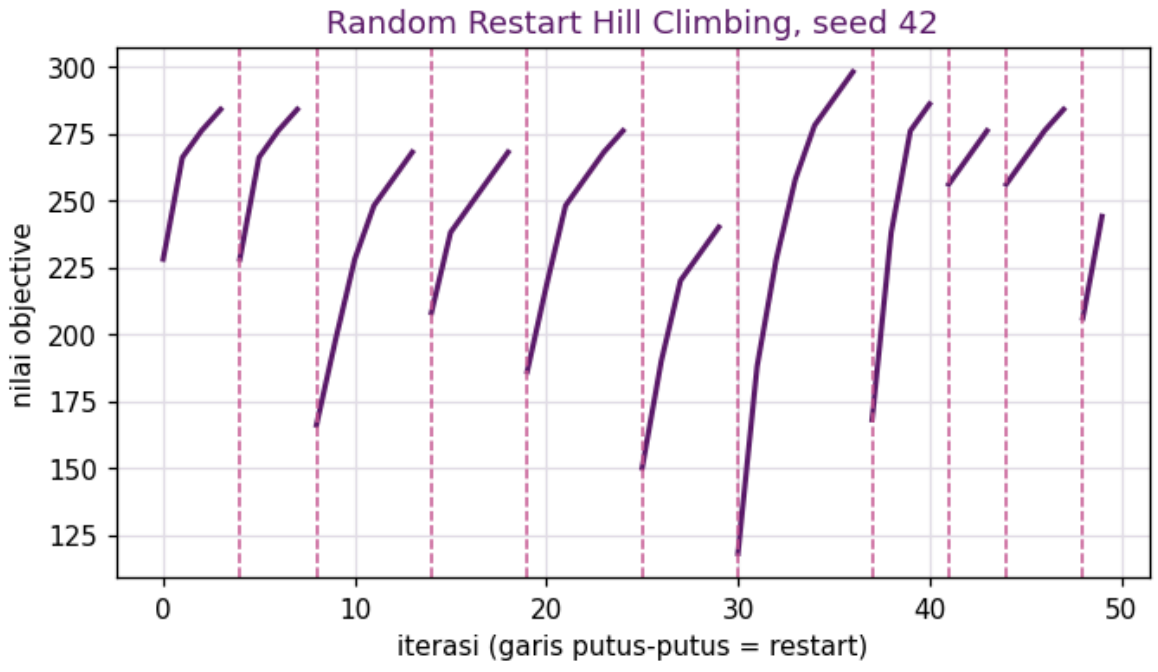
**Gambar 3.7** Visualisasi awal Random Restart Hill-Climbing



**Gambar 3.8** Visualisasi akhir Random Restart Hill-Climbing

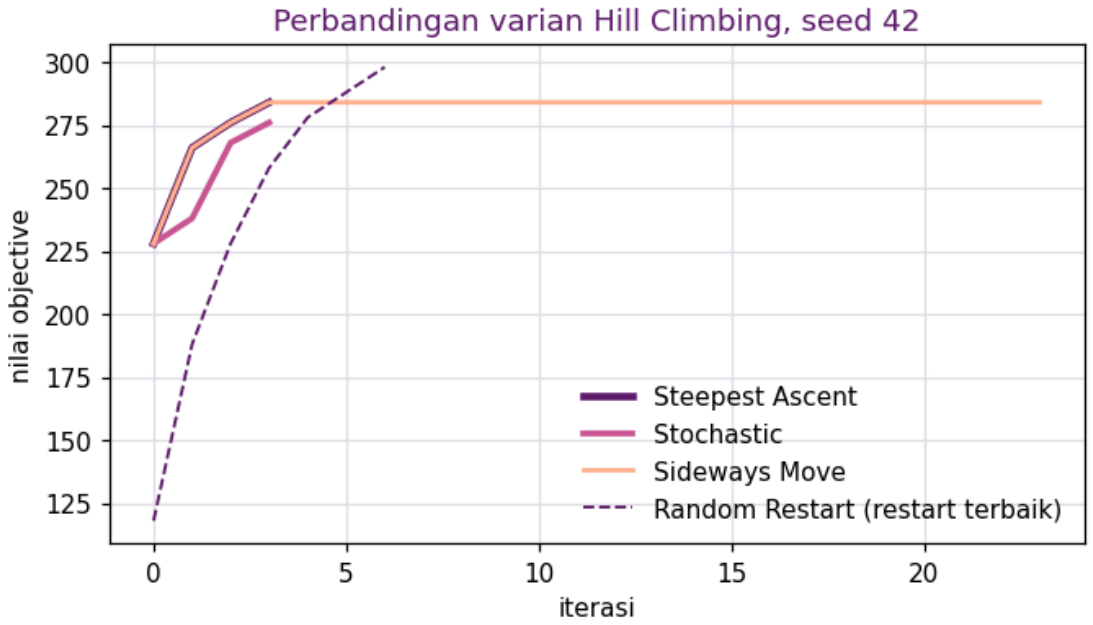



**Gambar 3.9** Grafik Random Restart Hill-Climbing



### 3.1.4 Analisis Perbandingan Variasi Hill-Climbing

**Gambar 3.10** Grafik perbandingan keempat varian Hill-Climbing




**Tabel 3.4 Ringkasan keempat varian Hill-Climbing**

| Varian | Nilai akhir | Rata-rata | Durasi rata-rata (detik) |
|---|---|---|---|
| Steepest Ascent | 284, 254, 284 | 274,0 | 0,07 |
| Stochastic | 276, 254, 284 | 271,3 | 0,08 |
| Sideways Move | 284, 286, 284 | 284,7 | 0,42 |
| Random Restart | 298, 298, 300 | 298,7 | 0,73 |

Random Restart Hill-Climbing memberikan hasil terbaik karena mencoba banyak titik awal dan mengambil hasil yang paling baik. Hasil satu kali pencarian bisa sangat berbeda tergantung titik awalnya, sehingga mencoba banyak titik awal sangat membantu. Sideways Move berada di urutan kedua. Pada percobaan 2, sideways move berhasil melewati daerah datar dan menaikkan nilai dari 254 menjadi 286, tetapi pada dua percobaan lainnya nilainya tidak berubah. Stochastic sedikit di bawah Steepest Ascent karena neighbor dipilih secara acak, sehingga kenaikannya tidak selalu yang paling besar. Dari sisi waktu, semua varian tetap cepat, tetapi Sideways Move dan Random Restart lebih lama karena jumlah iterasinya lebih banyak.

## 3.2 Objective Function ETA

### 3.2.1 Rancangan Objective Function ETA

*Objective function* tambahan ini menggunakan atribut ETA yang sudah ada, sehingga tidak menambah variabel baru sesuai ketentuan bonus. Kendaraan dengan ETA yang dekat sebaiknya diletakkan di dekat pintu masuk agar dapat diturunkan lebih dulu. Kendaraan dianggap mendesak jika ETA nya paling lama 3 hari, dan dianggap berada di zona depan jika seluruh badannya berada di setengah geladak yang dekat dengan pintu masuk. Jika kendaraan mendesak diletakkan di luar zona depan, ShippingFee kendaraan tersebut hanya dihitung setengahnya.

Objective function ini diimplementasikan pada fungsi value_eta. Fungsi dekat_ramp digunakan untuk memeriksa apakah kendaraan berada di zona depan, sedangkan fungsi count_salahzona digunakan untuk menghitung jumlah kendaraan mendesak yang berada di luar zona depan.

import library

```
import json

OUT = (-1, 0, 0, "H")
ETA_URGENT = 3
ETA_PENALTY = 0.5
```

dekat_ramp  dari file src/core.py

```
def dekat_ramp(problem, vehicle, placement):
    ship_idx, x, y, orientation = placement
    size = ukuran_vehicle(vehicle, orientation)
    if size is None or ship_idx < 0 or ship_idx >=
len(problem["ships"]):
        return False

    w, l = size
    ship_length = problem["ships"][ship_idx]["l"]
    front_limit = ship_length // 2
    return y + l <= front_limit
```

Value_eta dari file src/core.py

```
def value_eta(problem, state):
    vehicles = problem["vehicles"]
    total_score = 0
    for i in range(len(vehicles)):
```

```
        placement = state[i]
        if placement[0] == -1:
            continue

        vehicle = vehicles[i]
        fee = vehicle["fee"]
        if vehicle["eta"] <= ETA_URGENT and not dekat_ramp(problem,
vehicle, placement):
            fee = fee * (1 - ETA_PENALTY)
        total_score += fee
    return total_score
```

count_salahzona dari file src/core.py

```
def count_salahzona(problem, state):
    vehicles = problem["vehicles"]
    count = 0
    for i in range(len(vehicles)):
        placement = state[i]
        if placement[0] == -1:
            continue

        vehicle = vehicles[i]
        if vehicle["eta"] <= ETA_URGENT and not dekat_ramp(problem,
vehicle, placement):
            count += 1
    return count
```

### 3.2.2 Hasil dan Analisis

Objective function ETA diuji pada kasus satu kapal menggunakan keempat varian Hill-Climbing dan Genetic Algorithm.

**Tabel 3.5 Hasil objective function ETA**

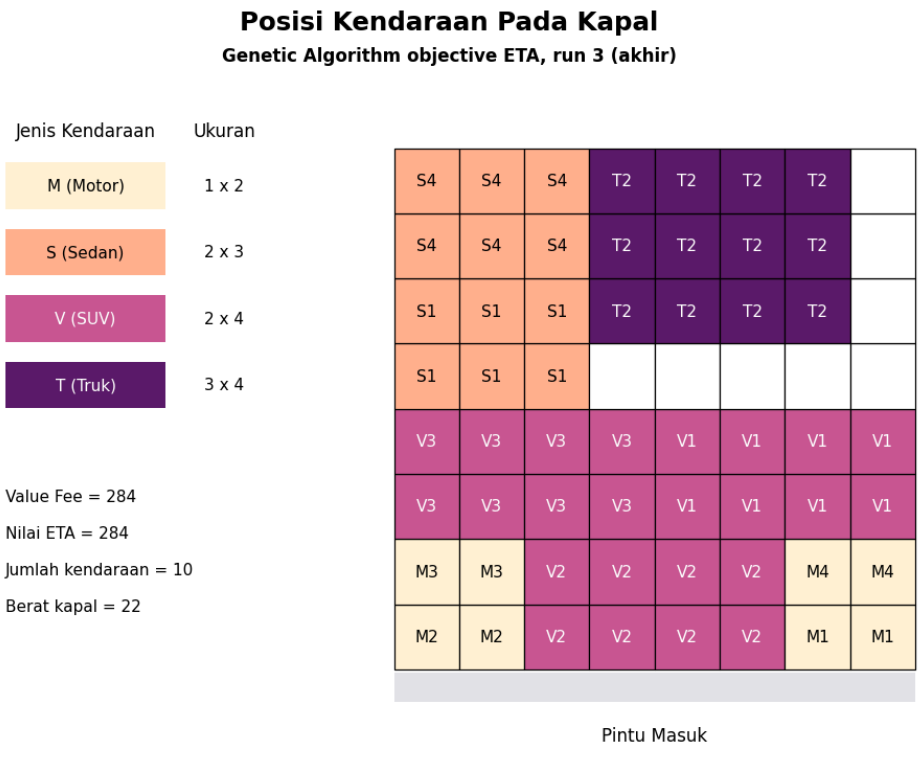
| Algoritma | Nilai akhir | Kendaraan di luar zona depan | Durasi Rata-rata (detik) |
|---|---|---|---|
| Steepest Ascent Hill-Climbing | 296, 238, 296 | 0, 0, 0 | 0,08 |
| Stochastic Hill-Climbing | 268, 238, 276 | 0, 0, 0 | 0,09 |
| Hill-Climbing with Sideways Move | 296, 284, 296 | 0, 0, 0 | 0,42 |
| Random Restart Hill-Climbing | 298, 298, 300 | 0, 0, 0 | 0,90 |

| | | | |
|---|---|---|---|
| Genetic Algorithm | 276, 278, 284 | 0, 0, 0 | 0,95 |

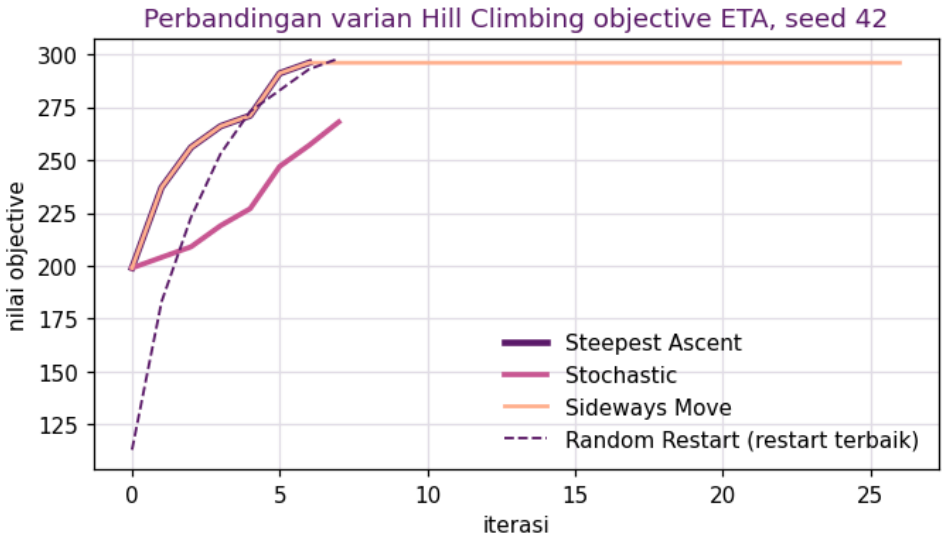
Pada semua percobaan, tidak ada kendaraan mendesak yang berada di luar zona depan. Artinya, objective function ETA berhasil mengarahkan algoritma untuk meletakkan kendaraan mendesak di dekat pintu masuk. Sebagai pembanding, dengan objective function ShippingFee masih terdapat beberapa kendaraan mendesak yang diletakkan jauh dari pintu masuk.

Pengaruhnya terhadap total ShippingFee berbeda pada setiap algoritma. Pada Genetic Algorithm, rata-rata ShippingFee turun sekitar 5% karena ruang di zona depan terbatas, sehingga beberapa kombinasi kendaraan yang menguntungkan tidak dapat digunakan. Pada Hill-Climbing, ShippingFee tidak selalu turun karena perubahan objective function juga mengubah arah pencarian dari initial state yang sama.

**Gambar 3.11 State akhir Genetic Algorithm dengan objective function ETA**



**Gambar 3.12 Grafik perbandingan varian Hill-Climbing dengan objective function ETA**




## 3.3 Multi-Kapal

### 3.3.1 Implementasi Multi-Kapal

Pengimplementasian multi-kapal tidak membutuhkan algoritma baru. Karena setiap posisi kendaraan sudah menyimpan nomor kapal, satu state dapat berisi kendaraan di kapal mana pun. Terdapat tiga bagian kode yang menyesuaikan dengan jumlah kapal.

- Fungsi `cek_placement` memeriksa sel dan berat setiap kapal secara terpisah.
- Fungsi `random_placement` memilih kapal secara acak sebelum memilih posisi.
- Fungsi `all_neighbors` mencoba semua posisi di semua kapal, sehingga kendaraan juga dapat dipindahkan dari satu kapal ke kapal lain.

Objective function yang digunakan tetap total ShippingFee dari seluruh kapal.

### 3.3.2 Hasil dan Analisis

Kasus multi-kapal diuji menggunakan keempat varian Hill Climbing, Simulated Annealing dan Genetic Algorithm.

**Tabel 3.6 Hasil kasus multi-kapal**

| Algoritma | Nilai akhir | Rata-rata | Durasi rata-rata (detik) |
|---|---|---|---|
| Simulated Annealing | 514, 512, 522 | 516,0 | 3,48 |
| Genetic Algorithm | 504, 504, 472 | 493,3 | 1,54 |

Simulated Annealing menghasilkan rata-rata yang lebih tinggi, yaitu sekitar 98% dari nilai optimum 524, sedangkan Genetic Algorithm sekitar 94%. Pada percobaan ketiga, Simulated Annealing bahkan sempat mencapai nilai optimum 524 selama proses, tetapi hasil akhirnya tetap state terakhir dengan nilai 522. Di sisi lain, Genetic Algorithm berjalan lebih cepat. Pada dua percobaan, Genetic Algorithm berhasil mengisi kedua kapal sampai kapasitas penuh, sedangkan pada percobaan ketiga masih ada sisa kapasitas sehingga nilainya lebih rendah. Durasi kedua algoritma lebih lama dibandingkan kasus satu kapal karena jumlah kendaraan lebih banyak dan pemeriksaan dilakukan pada dua kapal.

Keempat varian Hill-Climbing juga dijalankan pada kasus multi-kapal melalui program demo dengan seed 42, 43, dan 44 serta parameter yang sama dengan kasus satu kapal.

**Tabel 3.7 Hasil Hill-Climbing pada kasus multi-kapal**

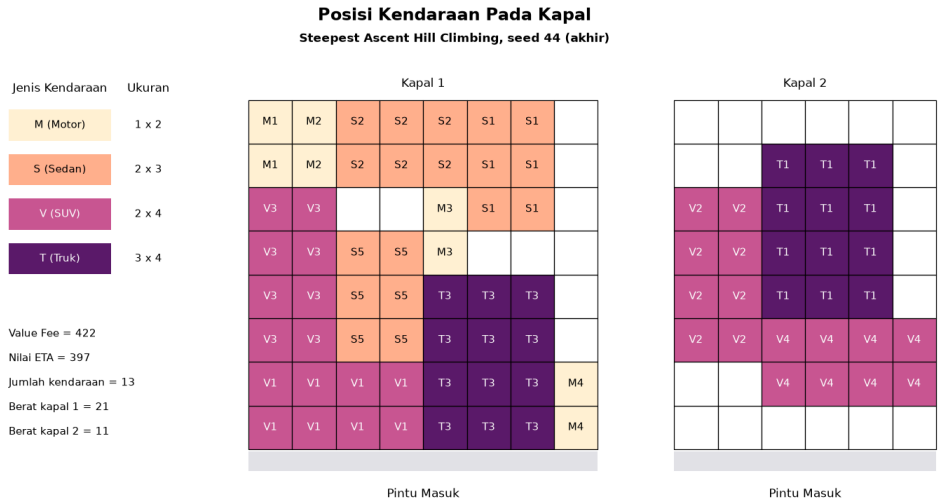
| Varian | Nilai akhir | Rata-rata | Iterasi | Durasi Rata-rata |
|---|---|---|---|---|
| Steepest Ascent | 444, 502, 442 | 462,7 | 5, 8, 9 | 0,74 |
| Stochastic | 422, 472, 440 | 444,7 | 6, 7, 10 | 0,66 |
| Sideways Move | 490, 502, 450 | 480,7 | 50, 28, 37 | 2,28 |
| Random Restart | 514, 516, 516 | 515,3 | 73, 79, 73 | 6,29 |

Random Restart Hill-Climbing memberikan hasil terbaik dengan rata-rata 515,3 atau sekitar 98% dari nilai optimum. Hasil ini hampir sama dengan Simulated Annealing dan lebih tinggi dari Genetic Algorithm. Pada dua percobaan, Random Restart mencapai nilai 516 dan mengisi kedua kapal sampai kapasitas penuh.

Steepest Ascent dan Stochastic Hill-Climbing tertinggal cukup jauh karena cepat berhenti di local optimum. Pada kasus multi-kapal, jumlah kendaraan dan kemungkinan susunan lebih banyak, sehingga local optimum yang bisa menjebak pencarian juga lebih banyak. Sideways Move sedikit lebih baik karena dapat melewati daerah datar, contohnya pada percobaan pertama yang naik hingga 490.

Durasi Hill-Climbing pada kasus ini lebih lama dibandingkan kasus satu kapal karena jumlah neighbor yang diperiksa di setiap iterasi jauh lebih banyak. Random Restart menjadi varian paling lama, yaitu sekitar 6 detik per percobaan, karena menjalankan Steepest Ascent sebanyak sebelas kali.

**Gambar 3.13 State akhir Hill-Climbing pada kasus multi-kapal**



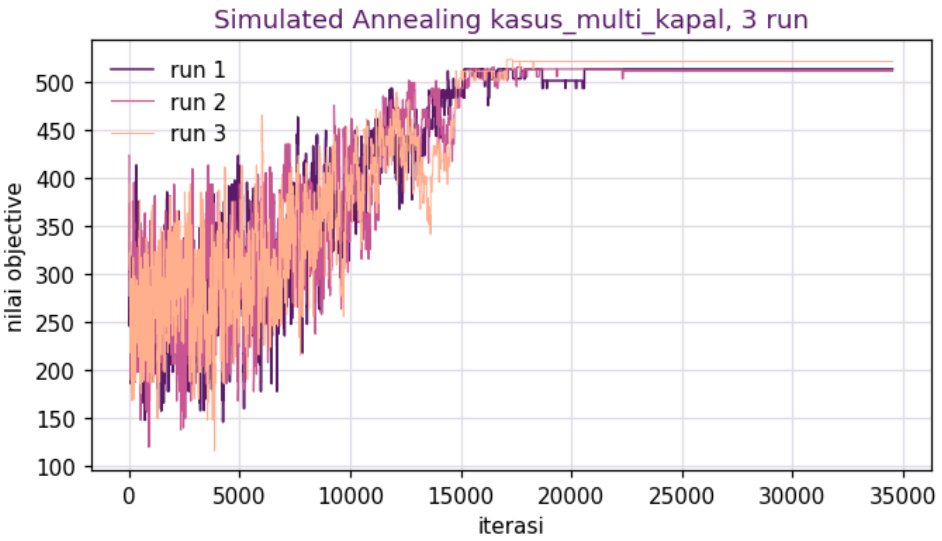

**Gambar 3.14 Grafik nilai objective function HC pada multi-kapal**



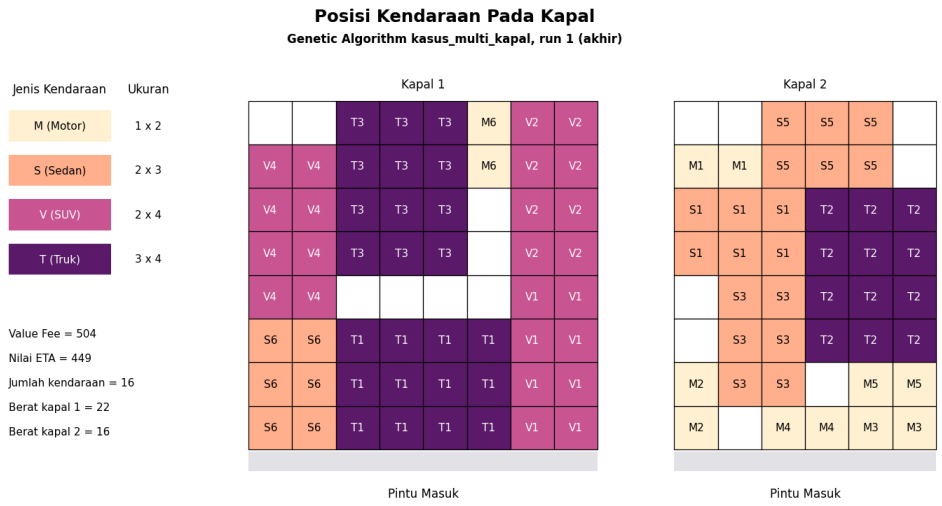
**Gambar 3.15 State akhir Simulated Annealing pada kasus multi-kapal**



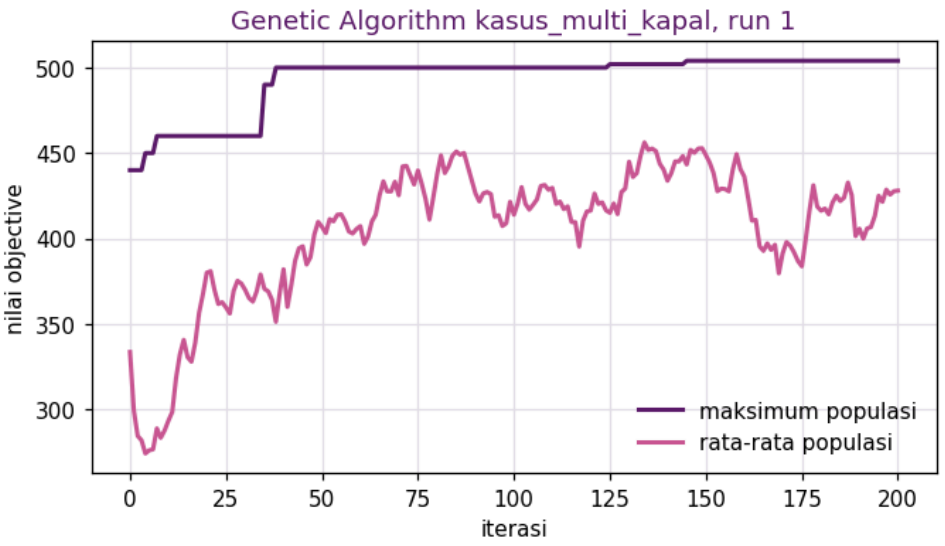

**Gambar 3.16 Grafik nilai objective function SA ketiga percobaan**



**Gambar 3.17 State akhir Genetic Algorithm pada kasus multi-kapal**




**Gambar 3.18 Grafik nilai objective function GA pada multi-kapal**



# BAB IV
# KESIMPULAN DAN SARAN

## 4.1 Kesimpulan

Dari tugas besar yang kami kerjakan, terdapat beberapa poin kesimpulan yang dapat ditarik, antara lain sebagai berikut.

1. Penataan kendaraan pada grid kapal sudah berhasil dimodelkan melalui metode local search, dengan state berupa informasi posisi dan orientasi yang dimiliki setiap kendaraan, dan state yang dihasilkan sudah pasti memenuhi constraint, yaitu tidak overlap, tidak keluar geladak, dan tidak melebihi kapasitas kapal.
2. Hill climbing merupakan algoritma yang paling cepat dijalankan, namun steepest ascent dan stochastic mudah berhenti di local optimum, sedangkan random restart menghasilkan value terbaik di antara keempat varian dengan rata-rata 298,7 karena bisa mencoba beberapa titik awal.
3. Simulated annealing menghasilkan value akhir 296 di ketiga run dan menjadi algoritma paling konsisten, tetapi membutuhkan iterasi yang jauh lebih banyak sehingga prosesnya paling lama dibandingkan algoritma lain.
4. Genetic algorithm menghasilkan rata-rata 294,9 dan menjadi satu-satunya algoritma yang hasil akhirnya mencapai nilai optimal 308.
5. Pada genetic algorithm, menambah jumlah populasi lebih berpengaruh dibanding menambah jumlah iterasi.
6. Objective function dengan variabel estimated time arrival (ETA) yang kami tambahkan untuk bonus berhasil menempatkan semua kendaraan mendesak di dekat pintu masuk, dengan konsekuensi total fee pada genetic algorithm turun sekitar 5% dibandingkan objective fee.
7. Pada kasus multi kapal, simulated annealing, dan random restart hill-climbing menghasilkan nilai yang tertinggi dengan rata-rata sekitar 516, lebih tinggi dibandingkan genetic algorithm, tetapi dengan jumlah iterasi yang jauh lebih banyak.

## 4.2 Saran

Terdapat beberapa saran yang kelompok kami ajukan untuk pengembangan dan improvisasi lanjutan dari tugas besar ini, yaitu:

1. Keenam algoritma sebaiknya dimulai dari state awal yang sama dan dijalankan lebih dari tiga kali agar lebih adil untuk dibandingkan dan hasil perbandingannya lebih terlihat.

2. Simulated annealing sebaiknya mengembalikan state terbaik yang pernah ditemukan, karena pada eksperimen ini simulated annealing sempat mencapai 308 tetapi hasil akhir tetap mengembalikan nilai 296.
3. Cara pemilihan induk pada genetic algorithm diubah supaya individu dengan nilai tinggi lebih sering terpilih, sehingga gen terbaik dapat lebih banyak diwariskan dan hasil terbaiknya dapat dicapai dengan langkah yang lebih sedikit.
4. Genetic algorithm bisa dihentikan lebih awal apabila terdapat kondisi nilai terbaiknya tidak berubah selama beberapa generasi, karena generasi yang ditambahkan tidak selalu memberikan jaminan memberi anak dengan gen yang lebih baik.
5. Pengujian bisa dilakukan pada kasus dengan skala yang lebih besar, misal kapal dengan grid lebih luas atau kendaraan yang lebih variatif dan jumlah banyak, agar perbedaan dan kelebihan tiap algoritma lebih terlihat.

# BAB V
# PEMBAGIAN TUGAS ANGGOTA KELOMPOK

Berikut merupakan pembagian kerja pada tugas besar.

**Tabel 5.1 Pembagian Tugas**

| Nama | NIM | Tugas |
|---|---|---|
| Indah Ramadhani | 18224037 | • Pemeriksaan constraint dan objective function<br>• Implementasi dan eksperimen keempat varian Hill-Climbing<br>• Bonus objective function ETA<br>• Laporan BAB I, subbab BAB II 2.1.3, 2.2, 2.4, BAB III 3.1, dan 3.2 |
| Tiara Clianta Andiwi | 18224078 | • Pembentukan initial state dan neighbor<br>• Implementasi dan eksperimen Simulated Annealing<br>• Bonus Multi-Kapal<br>• Laporan subbab BAB II 2.1.2, 2.1.4, 2.5, dan BAB III 3.3<br>• Form Penggunaan Generative AI<br>• Merapikan Laporan |
| Salsabila Shofiyah | 18224088 | • Dataset<br>• Implementasi dan eksperimen Genetic Algorithm,<br>• Visualisasi semua algoritma<br>• Program demo main.py<br>• README dan release<br>• Laporan BAB II subbab 2.1.1, 2.3, 2.6, 2.7, 2.8, BAB IV, dan BAB V |

# REFERENSI


[1] M. L. Khodra, “Beyond Classical Search: Classical vs Local Search,” YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=oGTa9kGE3P8. [Accessed: Sep. 16, 2026].

[2] M. L. Khodra, “Beyond Classical Search: State (Value, Neighbor),” YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM. [Accessed: Sep. 16, 2026].

[3] M. L. Khodra, “Beyond Classical Search: Hill-climbing Search,” YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Accessed: Sep. 16, 2026].

[4] M. L. Khodra, “Beyond Classical Search: Simulated Annealing,” YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=AdhG7HnhYrk. [Accessed: Sep. 16, 2026].

[5] M. L. Khodra, “Beyond Classical Search: Genetic Algorithm,” YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=aI41TGo9J9c. [Accessed: Sep. 16, 2026].

[6] J. D. Hunter, “Matplotlib: A 2D graphics environment,” Computing in Science & Engineering, vol. 9, no. 3, pp. 90-95, 2007, doi: 10.1109/MCSE.2007.55.

[7] simpleai contributors, “simpleai.search.local,” simpleai documentation. [Online]. Available: https://simpleai.readthedocs.io/en/latest/_modules/simpleai/search/local.html. [Accessed: Oct. 22, 2026].

[8] itsatefe, “Metaheuristic Algorithms for Bin Packing Problem (1D and 2D),” GitHub repository. [Online]. Available: https://github.com/itsatefe/Metaheuristic-Algorithms. [Accessed: Oct. 22, 2026].

[9] megics, “knapsack-GA,” GitHub repository. [Online]. Available: https://github.com/megics/knapsack-GA. [Accessed: Oct. 22, 2026].

# LAMPIRAN

## Lampiran A Form Penggunaan Generative AI

### FORMAT DEKLARASI PENGGUNAAN AI
### SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama / NIM : 1. Indah Ramadhani / 18224037
2. Tiara Clianta Andiwi / 18224078
3. Salsabila Shofiyah / 18224088

Fakultas / Sekolah : STEI-K

Kode Mata Kuliah : IF3070

Nama Mata Kuliah : Dasar Inteligensi Aritifisial

Judul Tugas/Proposal : Tugas Besar Dasar Inteligensi Artifisial

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

| Lingkup Pekerjaan | Digunakan? | Tingkat Penggunaan |
|---|---|---|
| **Pemeriksaan Ejaan:** menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya. | Ya | 1 - Pengecekan laporan |
| **Pembuatan Teks:** menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya. | Tidak | - |
| **Bantuan Diskusi dalam Penyusunan Konten:** menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya. | Ya | 2 - *AI-assisted idea generation and structuring* |
| **Pembuatan Gambar, Video, dan/atau Grafik:** menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya. | Tidak | - |
| **Lainnya, Jelaskan:**<br>Contoh penerapan kode dalam kasus lain sebagai *benchmark* (link github *benchmark* ada di *reference*). | Ya | 3 - *AI-assisted workflow* |

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026

| Indah Ramadhani | Tiara Clianta Andiwi | Salsabila Shofiyah |
|---|---|---|
| 18224037 | 18224078 | 18224088 |

## Lampiran B Tautan Repository GitHub

https://github.com/tiaracli/IF3070_G18_Triplets

## Lampiran C Bukti Pengerjaan

